VALOR Complete Technical Guide

Project VALOR: Complete Technical Guide & Engineering Reference

Project Title: SIH26181 — VALOR (Vital and Atmospheric Logic for Offline Rescue)
Hardware Platforms: 1. **Target Deployment:** Vicharak ShrikeFi (Espressif ESP32-S3 Dual-Core Xtensa LX7 + Renesas ForgeFPGA SLG47910C) 2. **Verified Golden Baseline:** AMD Xilinx Zynq-7000 SoC (`xc7z020clg400-1` Dual ARM Cortex-A9 + Artix-7 Fabric)
System Classification: Heterogeneous SoC — Synthesizable Verilog RTL + Dual-Core FreeRTOS C11 Firmware + Zero-Cloud INT8 TinyML Neural Network
Primary Mission: 100% Offline, Privacy-Preserving Personal Health Companion and Multi-Hazard Edge Disaster Monitor.

Table of Contents

1. Section A: Project Introduction
2. Section B: Technology Deep Dive from First Principles
3. Section C: System Architecture & Complete Data Flow
4. Section D: Complete Source-Code Walkthrough
5. Section E: Hardware & RTL Deep Dive (ForgeFPGA & Zynq)
6. Section F: AI/ML Pipeline & Zero-Cloud TinyML Engine
7. Section G: System Integration & Multicore Runtime Concurrency
8. Section H: Build, Simulation, Test, and Flashing Runbooks
9. Section I: Engineering Design Decisions & Trade-Off Matrix
10. Section J: Candid Limitations, Risk Register, and Future Roadmap
11. Section K: Glossary, Concept Dependency Map, and Pre-Judge Cheat Sheet

---

Section A: Project Introduction

1. What Valor Is

VALOR (Vital and Atmospheric Logic for Offline Rescue) is an edge-native, battery-powered medical and environmental monitoring wearable. It integrates synthesizable Verilog hardware acceleration with an ultra-low-power dual-core microcontroller and an on-device integer neural network. The device processes physiological signals from the human body in real time while simultaneously tracking atmospheric and environmental hazards, computing life-critical triage verdicts without relying on external servers, smartphones, or cellular infrastructure.

2. The Real-World Problem It Solves

During severe natural and industrial disasters—such as tropical cyclones, flash floods, heatwaves, structural collapses, and severe smog episodes—communication and power infrastructure are routinely obliterated: - **Cellular towers fail:** Backhaul fiber cuts, grid blackouts, and station immersion leave victims and search-and-rescue teams completely isolated from cloud services. - **Microclimate and physiological coupling:** Ambient air temperature, toxic particulate matter (PM2.5), and rapid barometric pressure drops directly exacerbate physiological distress. High heat causes cardiovascular strain and heat stroke; toxic particulates induce systemic inflammation and autonomic collapse; rapid barometric drops signal impending cyclones. - **Alarm fatigue in single-modality sensors:** Existing wearables monitor only vitals or only environmental metrics in isolation. An elevated heart rate (e.g., 135 BPM) during vigorous exertion in cool air is benign; that same heart rate at 44°C ambient temperature with high humidity represents impending heat stroke. Conventional fitness monitors cannot distinguish between these conditions, leading to dangerous alarm fatigue or unflagged fatalities.

3. Who Experiences the Problem

1. **Disaster First Responders (NDRF, SDRF, Fire Services):** Operating under intense physical strain in toxic, oxygen-depleted, or extreme heat conditions where autonomic exhaustion occurs rapidly.
2. **Evacuees and Isolated Populations:** Individuals trapped in cyclone shelters, flooded basements, or remote rural communities cut off from hospital emergency services.
3. **High-Risk Industrial Workers:** Miners, chemical plant workers, and construction crews exposed to toxic fumes, heat stress, and fall hazards.
4. **Rural Primary Healthcare Centers (PHCs):** Low-resource triage clinics lacking continuous bedside ICU monitors.

4. What Valor Does Differently

- **Zero Cloud Reliance:** Every physiological metric, signal filter, neural network inference, and triage rule runs 100% on bare metal silicon. If every satellite and cellular tower on Earth goes offline, Valor operates completely unaffected.
- **Hardware-Accelerated Optical Jitter Elimination:** An FPGA filter running at 50 MHz eliminates operating-system scheduling jitter, measuring inter-beat intervals (IBI) with nanosecond precision rather than the coarse, jittery 10 ms ticks of standard microcontroller RTOS loops.
- **Physiological-Environmental Sensor Fusion:** Integrates photoplethysmography (PPG), ambient barometric pressure trends, calibrated particulate scattering (PM2.5), ambient and skin temperature, relative humidity, 6-axis biomechanical motion (IMU), and satellite geolocation (GNSS).
- **Dual-Brain Architecture:** Strict separation between high-frequency signal conditioning (FPGA) and clinical/hazard triage (dual-core MCU).

5. Input, Processing, and Output Matrix

```
[Raw Physical Inputs]
  ├── Optical PPG (MAX30102 Red/IR @ 100 Hz)
  ├── Environmental Temp / Humidity / Pressure (BME280 @ 1 Hz)
  ├── Particulate Matter PM1.0 / PM2.5 / PM10 (Plantower PMS5003 UART @ 1 Hz)
  ├── 6-Axis Motion Ax, Ay, Az, Gx, Gy, Gz (MPU6050 IMU @ 40 Hz)
  ├── Global Satellite Geolocation NMEA (u-blox NEO-M8N / Quectel L89HS UART @ 1 Hz)
  └── Skin / Die Temperature (MAX30102 On-Chip Sensor)
        |
        ▼
[Hardware & Firmware Processing]
  ├── Renesas ForgeFPGA (50 MHz): 8-tap boxcar averaging + systolic peak detector (20 ns IBI)
  ├── ESP32-S3 Core 0: High-speed SPI acquisition + IBI ring buffer + Welford SpO2 engine
  ├── ESP32-S3 Core 1:
  |     ├── Signal Quality Index (Karlen SQI & motion gating)
  |     ├── Respiration from Autocorrelation (Charlton 2018 EDR)
  |    ├── Least-Squares Cyclone Barometric Trend (180-minute regression slope)
  |     ├── 3-Stage Biomechanical Fall Detection FSM (Free-fall -> Impact -
> Stillness)
  |     ├── Si et al. Neural PM2.5 Humidity Debiasing (INT8 MLP)
  |    ├── Moran's Physiological Strain Index (PSI) & AHA Autonomic Strain
  |     ├── Zero-Cloud INT8 Multi-Hazard Classifier (6->24->16->3 MLP)
  |    ├── Royal College of Physicians mNEWS2 Clinical Triage & Crisis Overrides
  |    └── Emergency SOS Latching State Machine (15s confirm, 5s cancel hold)
        |
        ▼
[Outputs & Rescuer Interfaces]
```

```
├── 0.96" I2C OLED (SSD1306): Live vitals & full-screen emergency medical card
├── Onboard Blue Heartbeat LED: Microsecond optical crest flash
├── Active Audio Sounder (GPIO 4): Fall alarm chirps & critical SOS siren
├── Local Wi-Fi SoftAP (192.168.4.1): Zero-install web dashboard & JSON status
└── Offline USB Telemetry Stream: Web Serial 60 FPS oscilloscope & Win32 GUI
```

6. Three Pitch Explanations

The 30-Second Elevator Pitch

> "Valor is an edge-native disaster triage wearable for extreme environments where communications and power grids are dead. It pairs a low-cost Renesas ForgeFPGA with a dual-core ESP32-S3 to capture heartbeats with microsecond precision and fuses raw vitals with ambient temperature, humidity, toxic smoke, cyclone pressure drops, and falls. Running an on-device integer neural network and clinical mNEWS2 protocols, Valor doesn't just tell a rescuer their heart rate is 140—it tells them whether they are suffering heat stroke, suffocating from toxic particulates, or hypothermic in floodwaters, alerting nearby teams with an offline emergency screen and local Wi-Fi dashboard."

The 2-Minute Technical Pitch

> "In disaster zones, cloud-dependent wearables fail because networks collapse, while MCU-only monitors fail because RTOS scheduling jitter destroys Heart Rate Variability (HRV) metrics. Valor solves this using a heterogeneous architecture.
>
> At the edge, raw optical PPG samples stream into a Renesas ForgeFPGA running at 50 MHz. Using an 8-tap running-sum boxcar filter and a 4-state systolic peak detector with a 250 ms refractory window, the FPGA detects systolic crests in pure digital logic with 0 DSP slices and 0 BRAM, eliminating timing jitter down to 20 nanoseconds.
>
> The ESP32-S3 reads these beat flags across an 8-bit SPI link on Core 0. Core 1 runs our clinical and environmental pipelines: Karlen Signal Quality Indexing, Welford-variance SpO2, Charlton respiratory rate extraction from pulse modulation, least-squares barometric regression for cyclone warnings, 3-stage fall detection, and an INT8 TinyML neural network that computes multi-hazard risk in under a microsecond. If vitals collapse, an autonomous SOS state machine triggers an audio siren, drives an OLED emergency card with GPS coordinates, and serves a zero-install diagnostic web dashboard from an onboard access point. It is completely private, requires zero internet, and operates entirely off a battery."

The Detailed Evaluator Explanation

> "Valor is a rigorously verified heterogeneous embedded system designed to meet Qualcomm SIH26181 requirements for an offline Personal Health Companion and Disaster Monitor.
>
> The system addresses physical layer, algorithmic, and architectural constraints: 1. **Timing Closure & Jitter:** The FreeRTOS 10 ms scheduling quantum on an ESP32-S3 introduces up to 10 ms of timestamp jitter—an error that swamps physiological RMSSD variations (which range between 10 and 50 ms). Valor offloads peak detection to synthesizable Verilog RTL. The baseline architecture was verified on an AMD Xilinx Zynq-7000 (`xc7z020`) achieving timing closure at 69.45 MHz (WNS +5.603 ns) using 185 LUTs and 0 DSPs. To reduce unit costs for field deployment, we ported this core to the Renesas ForgeFPGA SLG47910C, fitting into 363 LUT5s (32.41% fabric) with an Fmax of 82.73 MHz. 2. **Clinical Rigor:** Rather than treating health prediction as a black-box regression, Valor implements a dual-layer triage hierarchy. Deterministic medical protocols—the Royal College of Physicians Modified Early Warning Score (mNEWS2) and Si et al. humidity-calibrated PM2.5 models—govern absolute critical overrides (such as severe hypoxia SpO2 85% or tachycardia HR 140 BPM). In parallel, an on-device 6→24→16→3 INT8 multilayer perceptron evaluates non-linear multi-hazard stressors (heat exhaustion via Moran's PSI, smog distress, and flood hypothermia). 3. **Autonomous Operational Safety:** The system incorporates a state machine for emergency escalation (15-second confirmation to suppress false motion artifacts, 5-second hold-to-cancel, 30-second physiological recovery stand-down). When isolated, the device hosts an unencrypted local SoftAP web application directly from flash `.rodata`, allowing any rescuer with a standard smartphone browser to immediately access live physiological waveforms, 24-hour trend summaries, and GPS coordinates."

7. Implementation Status Matrix

Component / Subsystem	Implementation Status	Evidence / File Path
Zynq-7000 Baseline RTL	**Fully Implemented & Verified**	`hardware/zynq/tb_ppg_system.v` (6/6 tests passing, Vivado reports)
ForgeFPGA ShrikeFi RTL	**Fully Implemented & Verified**	`hardware/shrikefi/tb_forgefpga_system.v` (10/10 checks passing, 363 LUT5 log)

Component / Subsystem	Implementation Status	Evidence / File Path
SPI Target Interface	**Fully Implemented & Verified**	`hardware/shrikefi/spi_target.v`, `firmware/shrikefi/shrikefi_link_driver.c`
MAX30102 PPG & Skin Temp	**Fully Implemented & Verified**	`firmware/shrikefi/max30102.c` (100 Hz FIFO, on-chip die temp conversion)
BME280 / BMP280 Driver	**Fully Implemented & Verified**	`firmware/shrikefi/bme280.c` (Temp, press, hum with 50% neutral fallback)
Plantower PMS5003 UART	**Fully Implemented & Verified**	`firmware/shrikefi/pms5003.c` (UART1 @ 9600 baud, 32-byte frame validation)
MPU6050 IMU & Fall FSM	**Fully Implemented & Verified**	`firmware/shrikefi/mpu6050.c`, `fall_detect.c` (16/16 unit tests passing)
GNSS NMEA Parser	**Fully Implemented & Verified**	`firmware/shrikefi/gnss_nmea.c`, `gps_l89hs.c` (12/12 unit tests passing)
Welford SpO2 Engine	**Fully Implemented & Verified**	`firmware/core/spo2_engine.c` (50-sample variance ratio-of-ratios)
Karlen 2012 SQI Engine	**Fully Implemented & Verified**	`firmware/core/ppg_sqi.c` (Skewness, perfusion index, waveform entropy)
Charlton 2018 EDR (RR)	**Fully Implemented & Verified**	`firmware/core/ppg_respiratory_rate.c` (RSA IBI autocorrelation & tracking)
Barometric Storm Trend	**Fully Implemented & Verified**	`firmware/core/pressure_trend.c` (180-min least-squares slope regression)
Zero-Cloud INT8 TinyML	**Fully Implemented & Verified**	`firmware/core/nn_risk_model_int8.c` (619-byte MLP weights, host test verified)
mNEWS2 Clinical Triage	**Fully Implemented & Verified**	`firmware/core/clinical_vitals_engine.c` (Full score ladder & crisis overrides)
Autonomous SOS Engine	**Fully Implemented & Verified**	`firmware/shrikefi/sos.c` (IDLE/ARMED/ACTIVE/CANCELLED state machine)

Component / Subsystem	Implementation Status	Evidence / File Path
SSD1306 OLED Card	**Fully Implemented & Verified**	`firmware/shrikefi/ssd1306.c` (128x64 framebuffer, emergency card renderer)
Offline SoftAP Web UI	**Fully Implemented & Verified**	`firmware/shrikefi/web_status.c` (HTTPD daemon, `/status.json`, `/api/history`)
Win32 GDI GUI Dashboard	**Fully Implemented & Verified**	`firmware/shrikefi/shrikefi_dashboard.c` (Native Win32 C 60 FPS oscilloscope)
Web Serial HTML5 Dashboard	**Fully Implemented & Verified**	`firmware/shrikefi/dashboard/valor_dashboard.html` (Web Serial + simulated profiles)
MAX30205 Clinical Temp	*Architecture Ready / Auto-Fallback*	Integrated in `main_shrikefi.c` (probes 0x48; falls back to MAX30102 die temp)
Active Buzzer Hardware	*Pin Assigned (GPIO 4)*	Software state triggers exist; physical transistor circuit pending bench build
LoRa Mesh Broadcast	*Future Concept / Planned*	Protocol documented in `docs/REMAINING_WORK.md`; hardware not populated

---

Section B: Technology Deep Dive from First Principles

1. Field-Programmable Gate Array (FPGA)

- **What it is:** A semiconductor integrated circuit containing an uncommitted matrix of configurable logic blocks (CLBs), look-up tables (LUTs), flip-flops, and programmable routing channels whose electrical interconnections are configured after manufacturing via hardware description language (HDL) bitstreams.
- **Why it exists:** General-purpose CPUs execute software instructions se-

quentially through a fetch-decode-execute pipeline, introducing variable pipeline stalls, interrupt latency, and cache misses. FPGAs allow an engineer to synthesize custom digital circuits that execute operations in parallel with exact, clock-cycle-level determinism.

- **How it works:** Digital truth tables are mapped into SRAM-based Look-Up Tables (LUTs). For example, a 5-input LUT (LUT5) can implement any arbitrary Boolean function of 5 variables. Flip-flops register the outputs on clock edges. Dedicated programmable routing switches connect LUTs to form adders, shift registers, and state machines.
- **Why we use it in Valor:** Photoplethysmography (PPG) pulse analysis requires microsecond-accurate peak detection to calculate Heart Rate Variability (RMSSD). An operating system's software scheduler operates at millisecond resolution (e.g., 100 Hz = 10 ms tick). Running peak detection on an FPGA eliminates scheduling jitter entirely, providing 20 ns timing resolution at 50 MHz.
- **Implementation in Valor:** `hardware/common/moving_average_8tap.v`, `hardware/common/ppg_peak_detector.v`, `hardware/shrikefi/forgefpga_ppg_top.v`, and `hardware/shrikefi/spi_target.v`.
- **Alternatives & Trade-offs:**
  - *Dedicated Microcontroller Interrupts (GPIO EXTI):* Susceptible to ISR latency, FreeRTOS critical sections, and Wi-Fi radio DMA contention.
  - *Custom ASIC:* Prohibitive Non-Recurring Engineering (NRE) costs ($500k+) and zero post-manufacturing reconfigurability.
- **Limitations:** The Renesas ForgeFPGA SLG47910C is strictly constrained to 1120 LUT5s and lacks hardware DSP multiplier blocks and Block RAM (BRAM). Arithmetic must be constrained to additions, subtractions, and bitwise shifts.
- **Judge-Ready Explanation:** *"We use the FPGA as a specialized coprocessor for optical peak detection. Microcontroller operating systems introduce up to 10 milliseconds of scheduling jitter, which destroys heart rate variability measurements. Our FPGA implements an 8-tap filter and systolic peak detector directly in hardware, detecting peaks with 20-nanosecond resolution without consuming any CPU cycles."*

2. Dual-Core Xtensa LX7 Microcontroller (ESP32-S3)

- **What it is:** A 32-bit dual-core Harvard-architecture RISC processor featuring single-precision vector floating-point units (FPU), integrated 2.4 GHz Wi-Fi (802.11 b/g/n) and Bluetooth 5 (BLE), operating up to 240 MHz (configured in Valor at 160 MHz for thermal/power efficiency).
- **Why it exists:** High-level medical scoring, trigonometric floating-point regression, HTTP server stacks, and neural network inference require general-purpose sequential programming and standard C libraries that cannot fit onto ultra-low-density FPGAs.
- **How it works:** Core 0 (Protocol CPU / PRO_CPU) and Core 1 (Appli-

cation CPU / APP_CPU) operate concurrently under FreeRTOS, sharing 512 KB internal SRAM and accessing external flash memory via an internal cache bus.

- **Why we use it in Valor:** Core 0 is dedicated to microsecond-level SPI polling of the FPGA and I2C sensor servicing, shielding time-critical telemetry from interruption. Core 1 runs the compute-heavy clinical engines, TinyML integer neural network, and SoftAP HTTP server.
- **Implementation in Valor:** Configured via `firmware/shrikefi/sdkconfig.defaults` and initialized in `firmware/shrikefi/main_shrikefi.c`.
- **Alternatives & Trade-offs:**
  - *ARM Cortex-M4 (e.g., STM32F4):* Low power, but lacks integrated Wi-Fi for offline rescuer dashboards and requires separate radio modules.
  - *Single-Core Microcontroller:* Heavy floating-point calculations or web requests would freeze high-frequency sensor acquisition.
- **Limitations:** Single-precision hardware FPU only. Double-precision (`double`) operations are emulated in software, consuming hundreds of clock cycles. Shared memory between cores requires strict mutex protection to avoid data tearing.
- **Judge-Ready Explanation:** *"The ESP32-S3 acts as our high-level decision brain. By binding time-critical sensor acquisition to Core 0 and heavy clinical algorithms to Core 1, we ensure that generating an offline web dashboard or running neural network inference never delays optical sample processing."*

3. Photoplethysmography (PPG) & Inter-Beat Intervals (IBI)

- **What it is:** An optical technique that measures blood volume changes in the microvascular bed of tissue by illuminating the skin with Red (660 nm) and Infrared (880 nm) light and measuring the reflected or transmitted optical absorption.
- **Why it exists:** It provides non-invasive, continuous measurement of arterial oxygen saturation (SpO2), heart rate (HR), and autonomic nervous system state without requiring bulky, multi-lead electrocardiogram (ECG) chest electrodes.
- **How it works:** Oxygenated hemoglobin absorbs more infrared light, while deoxygenated hemoglobin absorbs more red light. Each systolic heart contraction pumps a pulse of blood through peripheral capillaries, creating a peak in optical absorption. The elapsed time between consecutive systolic peaks is the Inter-Beat Interval (IBI), measured in milliseconds.
- **Why we use it in Valor:** Serves as the primary physiological window into the patient. From raw PPG, Valor derives four distinct medical biomarkers: Heart Rate (HR), Blood Oxygenation (SpO2 via ratio-of-ratios), Heart Rate Variability (RMSSD via IBI differences), and Respiratory Rate (RR via respiratory sinus arrhythmia).

- **Implementation in Valor:** `firmware/shrikefi/max30102.c`, `firmware/core/spo2_engine.c`, `hardware/common/ppg_peak_detector.v`.
- **Alternatives & Trade-offs:**
  - *ECG (Electrocardiography):* Higher electrical fidelity, but requires conductive gel and electrodes that peel off in muddy, wet disaster environments.
  - *Phonocardiography (Acoustic):* Easily overwhelmed by ambient noise (wind, rain, heavy machinery).
- **Limitations:** Extremely sensitive to motion artifacts, ambient light leakage, and peripheral vasoconstriction during extreme hypothermia.
- **Judge-Ready Explanation:** *"PPG allows us to extract heart rate, blood oxygen, respiration, and autonomic stress from a single finger sensor. The challenge is motion noise, which we solve using our Karlen Signal Quality Index engine and FPGA systolic refractory filtering."*

4. Heart Rate Variability (HRV) & Root Mean Square of Successive Differences (RMSSD)

- **What it is:** The physiological phenomenon of variation in the time interval between consecutive heartbeats. RMSSD is the statistical root mean square of successive differences between adjacent normal-to-normal beat intervals:

$$\text{RMSSD} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N-1}(IBI_{i+1} - IBI_i)^2}$$

- **Why it exists:** The human heart is not a metronome. Heart rate is dynamically modulated beat-by-beat by the autonomic nervous system: the parasympathetic branch (vagus nerve) rapidly slows the heart, while the sympathetic branch accelerates it. High RMSSD indicates healthy autonomic resilience; depressed RMSSD (<15 ms) indicates severe physiological shock, systemic sepsis, exhaustion, or autonomic depression.
- **Why we use it in Valor:** Used by our Moran Physiological Strain Index (PSI) and TinyML models to detect heat exhaustion, physical collapse, and cardiopulmonary failure before gross vital signs fail.
- **Implementation in Valor:** `firmware/core/hrv_analysis.c` (`hrv_calculate_rmssd`).
- **Limitations:** Requires at least 10–30 clean, artifact-free consecutive intervals. Premature ventricular contractions (ectopic beats) distort RMSSD unless filtered out.
- **Judge-Ready Explanation:** *"RMSSD measures beat-to-beat variation in milliseconds. It reflects parasympathetic nervous system tone. When a first responder is suffering heat exhaustion, their RMSSD collapses minutes before their heart rate spikes, giving us an early-warning physiological indicator."*

5. **Modified Early Warning Score (mNEWS2)**

- **What it is:** An operationalized adaptation of the UK Royal College of Physicians National Early Warning Score 2 (NEWS2), a clinically validated bedside scoring system determining the degree of acute illness.
- **Why it exists:** Clinicians cannot monitor dozens of telemetry parameters simultaneously during mass-casualty triage. mNEWS2 maps physiological vitals into an aggregate risk integer (0 to 20+) and categorical triage levels (NORMAL, ELEVATED, HIGH, CRITICAL).
- **Why we use it in Valor:** Translates raw vitals into actionable clinical decisions on edge silicon without requiring doctor intervention. Valor implements both the continuous score ladder and absolute emergency overrides.
- **Implementation in Valor:** `firmware/core/clinical_vitals_engine.c` (`clinical_fuse_triage`).
- **Judge-Ready Explanation:** *"mNEWS2 is our clinical decision backbone. It standardizes heart rate, SpO2, respiration, and body temperature into an established score used in emergency rooms worldwide, backed by absolute overrides that flag life-threatening crises instantly."*

6. **Zero-Cloud TinyML (Integer Neural Networks)**

- **What it is:** Deep learning inference optimized for microcontrollers using 8-bit quantized weights and biases (INT8), running entirely in local SRAM without external coprocessors or runtime dependencies.
- **Why it exists:** Standard floating-point machine learning frameworks (TensorFlow, PyTorch) require megabytes of RAM and powerful operating systems. Edge microcontrollers require minimal memory footprints and integer-only arithmetic.
- **How it works:** Continuous floating-point weights and inputs are mapped onto signed 8-bit integers (`[-128, 127]`). Fixed-point scaling factors replace floating-point multiplications, and activations use piecewise linear approximations or lookup tables.
- **Why we use it in Valor:** Evaluates non-linear multi-hazard cross-correlations (e.g., combining high ambient temperature, high PM2.5, elevated HR, and suppressed RMSSD into distinct heat, smog, and flood risk probabilities) in a 619-byte footprint executing in 0.38 microseconds.
- **Implementation in Valor:** `firmware/core/nn_risk_model_int8.c`, trained via `firmware/core/train_nn_risk_model.py`.
- **Judge-Ready Explanation:** *"Our TinyML model is a 6-to-24-to-16-to-3 multi-layer perceptron compressed into 619 bytes of INT8 weights. It runs in under a microsecond on the microcontroller, predicting heat stroke, toxic inhalation, and hypothermia risks without touching the cloud."*

Section C: System Architecture & Complete Data Flow

```
flowchart TD
    subgraph SENSORS["1. Physical Transducers & Sensing Layer"]
        S_MAX["MAX30102<br/>Optical Red/IR LED + Photodiode<br/>(I2C @ 100 Hz, 0x57)"]
        S_BME["BME280 / BMP280<br/>Temp, Humidity, Barometric Press<br/>(I2C @ 1 Hz, 0x76)"]
        S_PMS["Plantower PMS5003<br/>Laser Optical Particle Counter<br/>(UART1 @ 9600 Baud)"]
        S_IMU["MPU6050 6-Axis IMU<br/>Accel (±8g) & Gyro (±1000 dps)<br/>(I2C @ 40 Hz, 0x68)"]
        S_GPS["u-blox NEO-M8N / L89HS<br/>NMEA-0183 GNSS Receiver<br/>(UART2 @ 9600 Baud)"]
    end

    subgraph HARDWARE["2. Hardware Acceleration Fabric (Renesas ForgeFPGA @ 50 MHz)"]
        HW_FILTER["moving_average_8tap.v<br/>8-Tap Running-Sum Boxcar Filter<br/>O(1) Arithmetic, Wir
        HW_PEAK["ppg_peak_detector.v<br/>Systolic 4-State FSM Peak Detector<br/>250 ms Refractory Bla
        HW_SPI["spi_target.v<br/>SPI Mode 0 Target Interface<br/>Frame: {beat, filt[6:0]}"]
        HW_LED["Direct Heartbeat Drive<br/>Blue LED (D12)"]
    end

    subgraph MCU_CORE0["3. ESP32-S3 Core 0 (Telemetry & Acquisition)"]
        C0_POLL["task_ppg_accelerator<br/>100 Hz Precision Acquisition Loop"]
        C0_SPI["SPI2 Master Driver<br/>Full-Duplex 1-Byte Transfers"]
        C0_IBI["IBI Timing & Ring Buffer<br/>1 µs Hardware Timer Latch"]
        C0_SPO2["spo2_engine.c<br/>Welford Running Variance Engine"]
    end

    subgraph MCU_CORE1["4. ESP32-S3 Core 1 (Processing, AI & Clinical Triage)"]
        C1_TASK["task_disaster_monitor<br/>1 Hz Master Clinical Evaluation"]
        C1_SQI["ppg_sqi.c<br/>Karlen 2012 SQI & Motion Gating"]
        C1_RR["ppg_respiratory_rate.c<br/>Charlton 2018 EDR via RSA"]
        C1_PTREND["pressure_trend.c<br/>180-min Least Squares Slope Engine"]
        C1_FALL["fall_detect.c<br/>3-Stage Biomechanical Fall FSM"]
        C1_GPS["gnss_nmea.c<br/>NMEA Latitude/Longitude Parser"]
        C1_ML["nn_risk_model_int8.c<br/>TinyML INT8 Multi-Hazard Classifier"]
        C1_NEWS["clinical_vitals_engine.c<br/>mNEWS2 Scoring & Crisis Overrides"]
        C1_SOS["sos.c<br/>Emergency State Machine Engine"]
    end

    subgraph OUTPUTS["5. Human & Machine Edge Interfaces"]
        O_OLED["SSD1306 128x64 OLED<br/>Live Vitals & Emergency Card"]
        O_BUZZ["Active Sounder (GPIO 4)<br/>Fall Chirps & Audio Siren"]
        O_AP["SoftAP Web Server (192.168.4.1)<br/>HTML5 Dashboard & JSON API"]
        O_USB["USB Telemetry Stream<br/>60 FPS Win32 & Web Serial GUI"]
    end
```

```
S_MAX -->|Raw Sample Byte| C0_POLL
C0_POLL -->|MOSI Sample| HW_SPI
HW_SPI --> HW_FILTER
HW_FILTER --> HW_PEAK
HW_PEAK -->|Crest Pulse| HW_LED
HW_PEAK -->|Beat Flag| HW_SPI
HW_SPI -->|MISO Byte| C0_SPI
C0_SPI --> C0_IBI
C0_POLL --> C0_SPO2

C0_IBI -->|Thread-Safe Mutex| C1_TASK
C0_SPO2 -->|Thread-Safe Mutex| C1_TASK
S_BME -->|I2C Polling| C1_TASK
S_PMS -->|UART1 Stream| C1_TASK
S_IMU -->|I2C 40 Hz| C1_FALL
S_GPS -->|UART2 NMEA| C1_GPS

C1_TASK --> C1_SQI
C1_TASK --> C1_RR
C1_TASK --> C1_PTREND
C1_TASK --> C1_ML
C1_TASK --> C1_NEWS
C1_FALL -->|Fall Event| C1_SOS
C1_NEWS -->|Critical Triage| C1_SOS

C1_TASK --> O_OLED
C1_SOS --> O_OLED
C1_SOS --> O_BUZZ
C1_TASK --> O_AP
C1_TASK --> O_USB
```

Complete End-to-End Physiological Data Trace

Here is the step-by-step trace of a single heartbeat through the entire system:

1. **Optical Acquisition:** The patient's finger rests on the MAX30102. At $t = 0$ ms, the MAX30102 fires its Red/IR LEDs and quantizes reflected light to an 18-bit ADC word. Core 0 reads this sample over I2C at 100 Hz (`max30102_read_fifo`).
2. **Dynamic Range Scaling:** Core 0 extracts an 8-bit dynamic-range scaled sample byte (`shrikefi_scale_sample`) and transmits it to the ForgeFPGA across the SPI2 bus at 1 MHz (`shrikefi_transceive_sample`).
3. **Hardware Digital Filtering:** The sample enters `moving_average_8tap.v`. The FPGA subtracts the sample from 8 clock cycles ago and adds the new sample to an internal 11-bit accumulator. The output is bit-shifted right by 3 (`>> 3`), producing a low-pass filtered sample within a single 20 ns

clock cycle.

4. **Systolic Peak Detection:** The filtered sample enters `ppg_peak_detector.v`. When the sample rises above the threshold (120) and subsequently drops by 3 units (`MIN_FALL_STEP`), the FSM confirms a true systolic crest, sets the beat flag, asserts the blue heartbeat LED (D12), and arms the 250 ms refractory timer.
5. **SPI Return Transfer:** On the rising edge of `spi_ss_n`, `spi_target.v` latches `{beat, filt[6:0]}` into its output register, returning it to the ESP32-S3 over MISO.
6. **Microsecond Inter-Beat Interval Timing:** Core 0 inspects bit 7. Detecting a beat flag, it captures the hardware timer (`esp_timer_get_time()`), calculates the IBI as $\Delta t = t_{now} - t_{last}$, and pushes the interval into an 8-beat circular buffer.
7. **Cross-Core Memory Transfer:** Core 0 writes the new heart rate, IBI, and Welford SpO2 into `g_state` under the protection of `s_data_mutex`.
8. **Clinical Evaluation on Core 1:** At the 1 Hz evaluation boundary, Core 1 acquires `s_data_mutex`.
   - `ppg_sqi.c` verifies signal quality: if $SQI \geq 0.70$, vitals are marked trustworthy.
   - `ppg_respiratory_rate.c` runs autocorrelation across the IBI series, extracting the breathing rate from respiratory sinus arrhythmia.
   - `pressure_trend.c` checks the rolling barometric slope.
   - `nn_risk_model_int8.c` normalizes vitals and environmental metrics into INT8 features and runs forward inference in 0.38 µs.
   - `clinical_vitals_engine.c` evaluates mNEWS2 criteria and flags critical overrides.
9. **Emergency Escalation:** If mNEWS2 indicates CRITICAL, `sos.c` starts the 15-second confirmation counter. If sustained, state latches to `ACTIVE`.
10. **Actuation & Telemetry Output:**

- The SSD1306 OLED displays the emergency card: `* MEDICAL EMERGENCY *`, current vitals, condition, and GPS location.
- The active buzzer sounds an audible alarm pattern.
- `web_status.c` updates `/status.json`, turning the local SoftAP web page crimson with emergency instructions.
- Serial telemetry prints `[TRIAGE]` and `[TELEMETRY]` frames to USB for PC dashboard visualization.

Section D: Complete Source-Code Walkthrough

1. Repository File Inventory & Responsibility Map

```
FPGA-Accelerated-Personal-Health-Companion/
├── hardware/
```

```
│   ├── common/
│   │   ├── moving_average_8tap.v               #  Synthesizable  8-
tap boxcar filter (vendor-agnostic)
│   │   ├── ppg_peak_detector.v       # 4-state systolic peak detector with refractory FSM
│   │   └── tb_ppg_peak_detector.v     # Self-checking testbench for peak detector
│   ├── shrikefi/
│   │   ├── forgefpga_ppg_top.v       # Top-level RTL for Renesas ForgeFPGA SLG47910C
│   │   ├── spi_target.v                # SPI Mode 0 target interface (8-
bit full duplex)
│   │   ├── forgefpga_pins.pcf        # Physical pin constraints for ForgeFPGA
│   │   ├── tb_forgefpga_system.v      # 10-test comprehensive RTL simulation suite
│   │   └── synthesis_evidence/       # Verified fitter reports & utilization logs
│   └── zynq/
│           ├── axi_ppg_accelerator.v             #  Memory-mapped  AXI4-
Lite slave wrapper for Zynq PS
│           ├── tb_ppg_system.v                      #  6/6  test  self-
checking baseline testbench
│   └── test_disaster_risk_engine.c  # Master host regression test harness (C11)
├── firmware/
│   ├── core/
│   │   ├── clinical_vitals_engine.c/.h  # mNEWS2 clinical triage & emergency overrides
│   │   ├── disaster_risk_engine.c/.h    # Multi-hazard physiological/environmental rule engine
│   │   ├── hrv_analysis.c/.h         # Statistical RMSSD and SDNN HRV calculation
│   │   ├── nn_risk_model_int8.c/.h     # Zero-cloud INT8 TinyML neural network inference
│   │   ├── pm25_calibration_int8.c/.h   # Si et al. INT8 neural humidity PM2.5 debiasing
│   │   ├── ppg_respiratory_rate.c/.h    # Charlton 2018 EDR respiratory rate estimator
│   │   ├── ppg_sqi.c/.h            # Karlen 2012 Signal Quality Index (entropy/skewness)
│   │   ├── pressure_trend.c/.h        # Least-squares barometric slope cyclone regression
│   │   └── spo2_engine.c/.h          # Welford running variance SpO2 ratio-
of-ratios
│   └── shrikefi/
│       ├── main_shrikefi.c          # Dual-core FreeRTOS application entry point
│       ├── max30102.c/.h                # MAX30100/MAX30102 driver & on-
chip die temperature
│       ├── bme280.c/.h            # BME280/BMP280 pressure, temp & humidity driver
│       ├── pms5003.c/.h           # Plantower PMS5003 UART particulate driver
│           ├── mpu6050.c/.h                                 #  MPU6050  6-
axis IMU driver (accelerometer/gyroscope)
│       ├── fall_detect.c/.h         # 3-stage biomechanical fall detection FSM
│       ├── gnss_nmea.c/.h                  # NMEA-0183 GNSS parser (u-
blox & Quectel)
│       ├── gps_l89hs.c/.h                # Quectel L89HS / u-blox NEO-
M8N hardware interface
│       ├── ssd1306.c/.h            # SSD1306 128x64 OLED display driver & framebuffers
│       ├── sos.c/.h                    # Emergency assistance state machine
│       ├── location.c/.h           # Stored NVS location manager & sanitization
```

```
|     ├── web_status.c/.h          # SoftAP HTTP web daemon, `/status.json`, `/api/history`
|     ├── shrikefi_link_driver.c/.h   # SPI2 master driver & bitstream programmer
|     ├── forgefpga_bitstream.h      # Pre-compiled ForgeFPGA bitstream C array
|       └── dashboard/
|         └── valor_dashboard.html    # HTML5/JS Web Serial 60 FPS GUI dashboard
└── docs/                         # Architectural specifications & runbooks
```

2. Deep Dive: Firmware Core Modules

A. Clinical Vitals Engine (`firmware/core/clinical_vitals_engine.c`)

- **Purpose:** Implements the Royal College of Physicians mNEWS2 scoring matrix alongside deterministic medical override rules.
- **Key Functions:**
  - `clinical_score_news2()`: Scores Heart Rate (HR), SpO2, Respiratory Rate (RR), and Temperature against clinical lookup bands.
  - `clinical_fuse_triage()`: Integrates vital scores, checks Signal Quality Index (SQI) motion flags, and checks for absolute emergency overrides:
    - Hypoxia Override: `SpO2 <= 85%` → CRITICAL (Alert flag `0x01`).
    - Tachycardia Override: `HR >= 140 bpm` → CRITICAL (Alert flag `0x02`).
    - Bradycardia Override: `HR <= 35 bpm` → CRITICAL (Alert flag `0x04`).
    - Bradypnea Override: `RR <= 6 br/min` → CRITICAL (Alert flag `0x20`).
    - Tachypnea Override: `RR >= 30 br/min` → CRITICAL (Alert flag `0x10`).
    - Hypothermia Override: `Temp <= 35.0°C` → CRITICAL (Alert flag `0x80`).
- **Motion Gating Logic:** If `sqi < 0.70`, standard mNEWS2 triage is suspended with status `SENSOR QUALITY LOW (holding triage)`, preventing motion-induced false alarms. However, if an absolute crisis override is triggered, the emergency alarm fires regardless of noise, prioritizing patient safety.

B. SpO2 Engine (`firmware/core/spo2_engine.c`)

- **Purpose:** Computes arterial blood oxygen saturation (SpO_2) using the ratio-of-ratios equation without relying on vendor-proprietary, closed-source libraries:

$$R = \frac{AC_{red}/DC_{red}}{AC_{ir}/DC_{ir}} = \frac{\sigma_{red}/\mu_{red}}{\sigma_{ir}/\mu_{ir}}$$

- **Implementation:** Uses Welford's algorithm to compute running mean (μ) and variance (σ^2) across a 50-sample rolling window in a single pass with numerical stability.

- **Validation Guard:** Requires 8 consecutive valid 50-sample windows and verifies that AC pulsation is between 0.1% and 10% of DC to reject ambient air or non-tissue contact before latching a valid SpO2 reading.

C. Charlton Respiratory Rate Estimator (`firmware/core/ppg_respiratory_rate.c`)

- **Purpose:** Extracts breathing rate from raw Inter-Beat Intervals by tracking Respiratory Sinus Arrhythmia (RSA)—the natural modulation of heart rate during inhalation and exhalation.
- **Implementation:**
  1. Detrends the recent 30-beat IBI series using a linear least-squares fit to remove baseline physiological drift.
  2. Computes the normalized autocorrelation function across lags of 2 to 12 beats.
  3. Uses parabolic interpolation around the correlation peak to resolve fractional lags, eliminating integer grid quantizations.
  4. Applies harmonic suppression: selects the lowest local maximum reaching 60% of the dominant peak to prevent octave errors (reporting half the true breathing rate).
  5. Implements a strict lag-2 rejection filter (`PPG_RR_MIN_LAG = 3`) to prevent beat alternans artifacts from registering as false tachypnea.

D. Barometric Cyclone Slope Engine (`firmware/core/pressure_trend.c`)

- **Purpose:** Evaluates atmospheric pressure drop rates to provide early warnings for developing cyclones and severe gales.
- **Mathematical Mechanism:** Stores up to 180 barometric pressure readings (sampled once per minute, representing a 3-hour history) in a circular buffer. Computes the least-squares regression slope:

$$\text{Slope} = \frac{N\sum(t \cdot P) - \sum t \sum P}{N\sum(t^2) - (\sum t)^2} \text{ [hPa/hr]}$$

- **Thresholds & Drop Floor:**
  - ≤ -0.5 hPa/hr: MODERATE (unsettled weather).
  - ≤ -1.0 hPa/hr: HIGH (developing storm/gale).
  - ≤ -2.0 hPa/hr: CRITICAL (rapidly deepening cyclone).
  - **Load-Bearing Guard:** Requires a minimum absolute drop of at least 0.5 hPa over the window to ensure sensor thermal jitter does not trigger false storm alerts.

E. Fall Detection FSM (`firmware/shrikefi/fall_detect.c`)

- **Purpose:** Detects catastrophic human falls using a 3-stage biomechanical finite state machine on 40 Hz IMU data.
- **State Machine Stages:**
  1. `MONITORING`: Continuous baseline tracking.

2. `FREE_FALL`: Accelerometer magnitude $A_{mag} = \sqrt{Ax^2 + Ay^2 + Az^2} < 0.6g$ for ≥ 60 ms.
3. `IMPACT`: Acceleration spike $> 2.5g$ accompanied by rotational velocity $\omega > 120^{\circ}/\text{s}$.
4. `INACTIVITY`: Immobility window ($A_{mag} \in [0.75g, 1.25g]$ and motion variance $< 0.15g$) sustained for 1.5 seconds.
5. `FALL_CONFIRMED`: Posture angle change $> 45^{\circ}$ verified relative to pre-fall vertical. Arms the emergency SOS system with a 10-second cancellation window.

Section E: Hardware & RTL Deep Dive (ForgeFPGA & Zynq)

1. RTL Fundamentals & Coding Contracts

All RTL modules in `hardware/common/` adhere to strict synthesis contracts: - **Vendor-Agnostic Design:** Zero vendor primitives, zero AXI bindings, zero dedicated DSP/BRAM primitives. Compiles identically in Vivado ML, Renesas ForgeFPGA Workshop, and Icarus Verilog. - **Clocking & Resets:** Active-high synchronous reset (`posedge clk`) across all internal registers. - **Assignment Discipline:** Non-blocking assignments (`<=`) in sequential blocks; continuous assignments (`assign`) for combinational datapath outputs. Zero latch inference. - **Arithmetic Constraints:** All scaling performed using power-of-two division (`>> 3`), mapped directly to hardware wire re-indexing without generating logic gates or multiplier circuits.

2. Module 1: 8-Tap Running-Sum Moving Average Filter (`moving_average_8tap.v`)

```
module moving_average_8tap (
    input  wire       clk,
    input  wire       rst,
    input  wire [7:0] i_sample,
    output wire [7:0] o_filtered
);
```

Datapath Architecture & Theory of Operation A conventional finite impulse response (FIR) filter computes:

$$y[n] = \frac{1}{8} \sum_{k=0}^{7} x[n-k]$$

Directly summing 8 samples every clock cycle requires an adder tree of 7 adders. The `moving_average_8tap` module implements an $O(1)$ recursive running-sum

algorithm:

$$S[n] = S[n-1] + x[n] - x[n-8], \quad y[n] = \lfloor S[n]/8 \rfloor$$

Hardware Registers & State Elements

- `reg [7:0] shift_reg [0:7]`: An 8-deep array of 8-bit registers forming a shift-register delay line.
- `reg [10:0] sum`: An 11-bit accumulator. Because $8 \times 255 = 2040$, 11 bits ($2^{11} = 2048$) prevent arithmetic overflow.

Cycle-by-Cycle Execution Trace On every positive clock edge (`posedge clk`):

```
always @(posedge clk) begin
    if (rst) begin
        sum <= 11'd0;
        // clear shift register array...
    end else begin
        sum <= sum + i_sample - shift_reg[7];
        shift_reg[0] <= i_sample;
        shift_reg[1] <= shift_reg[0];
        // ... propagate down to shift_reg[7]
    end
end
assign o_filtered = sum[10:3]; // Hardwired truncation (division by 8)
```

- **Cycle 0:** Reset asserted → `sum = 0`, `shift_reg = [0, 0, 0, 0, 0, 0, 0, 0]`.
- **Cycle 1:** Stimulus `i_sample = 80`. `sum <= 0 + 80 - 0 = 80`. `shift_reg[0] <= 80`. `o_filtered = 80 >> 3 = 10`.
- **Cycle 8:** Shift register fully populated with 80. `sum = 640`. `o_filtered = 640 >> 3 = 80`. Filter converged.

3. Module 2: Systolic PPG Peak Detector (`ppg_peak_detector.v`)

```
module ppg_peak_detector #(
    parameter THRESHOLD         = 8'd120,
    parameter REFRACTORY_CYCLES = 32'd12500000, // 250 ms @ 50 MHz
    parameter MIN_FALL_STEP     = 8'd3          // H-01 slope gate
)(
    input  wire        clk,
    input  wire        rst,
    input  wire [7:0]  i_sample,
    output reg         o_beat,
    output reg [31:0]  o_ibi_cycles
);
```

The 4-State Finite State Machine (FSM) To reject dicrotic notches, baseline wander, and optical noise spikes, the detector executes a 4-state Mealy/Moore FSM:

```
stateDiagram-v2
    [*] --> STATE_IDLE
    STATE_IDLE --> STATE_ARMED: i_sample >= THRESHOLD (120)
    STATE_ARMED --> STATE_CREST_FALL: i_sample < peak_val
    STATE_ARMED --> STATE_ARMED: i_sample >= peak_val (update peak_val)
    STATE_CREST_FALL --> STATE_REFRACTORY: (peak_val - i_sample) >= MIN_FALL_STEP (3) [EMIT BEAT]
    STATE_CREST_FALL --> STATE_ARMED: i_sample > prev_sample (re-climbing)
    STATE_REFRACTORY --> STATE_IDLE: refractory_cnt >= 12,500,000 (250 ms)
```

1. **`STATE_IDLE (2'b00):`** System waits for optical pulse amplitude to cross the detection threshold (`i_sample >= 120`). Sub-threshold noise cannot arm the detector.
2. **`STATE_ARMED (2'b01):`** Signal has crossed threshold. While the signal continues rising, `peak_val` tracks the maximum sample value. Once `i_sample < peak_val`, the signal has crested, transitioning to `STATE_CREST_FALL`.
3. **`STATE_CREST_FALL (2'b10):`** The H-01 slope gate. Verifies that the signal drops by at least `MIN_FALL_STEP` (3 units). This prevents a flat noise plateau at the top of a wave from triggering multiple false beats. Once the drop is confirmed:
   - `o_beat <= 1'b1` (fires for exactly 1 clock cycle).
   - `o_ibi_cycles <= ibi_counter` (latches the 50 MHz cycle counter).
   - `ibi_counter <= 32'd0` (resets interval timer).
   - Transitions to `STATE_REFRACTORY`.
4. **`STATE_REFRACTORY (2'b11):`** Refractory blanking window. Biological human heart rates cannot exceed 240 BPM (a minimum interval of 250 ms). The FSM holds in this state for 12,500,000 clock cycles (250 ms at 50 MHz), completely ignoring the physiological dicrotic notch before returning to `STATE_IDLE`.

4. Module 3: SPI Target Interface (`spi_target.v`)

- **Protocol:** SPI Mode 0 (`CPOL=0, CPHA=0`).
- **Signal Contract:**
  - `i_sck`: SPI Clock driven by ESP32-S3 (1 MHz runtime).
  - `i_ss_n`: Active-low Chip Select driven manually by ESP32-S3 GPIO 10.
  - `i_mosi`: Master-Out Slave-In (transfers raw sample byte from MCU to FPGA).
  - `o_miso`: Master-In Slave-Out (transfers `{beat, filt[6:0]}` from FPGA to MCU).
- **Reload Mechanism:** The shift register reloads its transmission byte **only on the rising edge of `i_ss_n`**. Between transactions, `i_ss_n` must return high for at least 40 ns (2 clock cycles at 50 MHz) to prepare the

next frame.

5. Physical Synthesis & Resource Utilization

Renesas ForgeFPGA SLG47910C (ShrikeFi Target):

- **Toolchain:** Renesas ForgeFPGA Workshop v6.55.
- **Evidence Log:** `hardware/shrikefi/synthesis_evidence/resource_utilization_spi_link.log` (timestamped 2026-09-21).
- **Physical Silicon Metrics:**
  - **CLB LUT5s: 363 / 1120 (32.41%)** (all used as logic; 0 shift-register LUTs).
  - **Flip-Flops: 202** (198 CLB FFs [17.68%] + 4 IOB FFs [0.54%]).
  - **CLB Blocks Occupied: 75 / 140 (53.57%)**.
  - **4k BRAMs: 0 / 8 (0.00%)**.
  - **PLLs: 0 / 1 (0.00%)** (operates from internal ring oscillator).
  - **Timing Closure:** Fmax = **82.73 MHz** (achievable period 12,087 ps, providing +7.913 ns slack against 50 MHz target).

AMD Xilinx Zynq-7000 (`xc7z020clg400-1` Verified Baseline):

- **Toolchain:** AMD Vivado ML 2022.2 out-of-context synthesis.
- **Evidence Log:** `hardware/zynq/synthesis_report.txt`.
- **Physical Silicon Metrics:**
  - **Slice LUTs: 185 / 53,200 (0.35%)**.
  - **LUTRAMs: 16 / 17,400 (0.09%)**.
  - **Flip-Flops: 266 / 106,400 (0.25%)**.
  - **DSP48 Multipliers: 0 / 220 (0.00%)**.
  - **Block RAM: 0 / 140 (0.00%)**.
  - **Timing Closure:** WNS = **+5.603 ns**, WHS = **+0.184 ns**, Fmax = **69.45 MHz** against 50 MHz constraint.

Section F: AI/ML Pipeline & Zero-Cloud TinyML Engine

1. Model Architecture & Parameters

Valor deploys an ultra-compact, zero-cloud feed-forward multi-layer perceptron (MLP) for multi-hazard physiological and environmental risk prediction: - **Topology:** Fully Connected 6 $\rightarrow$ 24 $\rightarrow$ 16 $\rightarrow$ 3 Architecture. - **Layer 1 (Input):** 6 features. - **Layer 2 (Hidden):** 24 neurons, ReLU activation. - **Layer 3 (Hidden):** 16 neurons, ReLU activation. - **Layer 4 (Output):** 3 neurons, Sigmoid activation. - **Parameter Count:** - Weights: $(6 \times 24) + (24 \times 16) + (16 \times 3) = 144 + 384 + 48 = 576$ parameters. -

Biases: $24 + 16 + 3 = 43$ parameters. - Total Parameters: **619 bytes** in 8-bit quantization (`int8_t`).

2. Input Features & Preprocessing Pipeline

Every second, Core 1 constructs a 6-element normalized feature vector:

Index	Feature	Source Sensor	Normalization Mapping	Range
x_0	**Heart Rate (HR)**	MAX30102 via FPGA	$(HR - 30.0)/190.0$	$30 - 220$ BPM
x_1	**Autonomic Tone (RMSSD)**	IBI Variance Engine	$\text{clamp}(RMSSD/100.0, 0.0, 1.0)$	$0 - 100$ ms
x_2	**Oxygen Saturation (SpO2)**	Welford Variance Engine	$(SpO2 - 70.0)/30.0$	$70 - 100\%$
x_3	**Ambient Temperature**	BME280 Environment	$(T_{amb} - 0.0)/55.0$	$0 - 55°\text{C}$
x_4	**Relative Humidity**	BME280 Environment	$RH/100.0$	$0 - 100\%$
x_5	**Calibrated PM2.5**	PMS5003 + Si et al. NN	$\text{clamp}(PM2.5/500.0, 0.0, 1.0)$	$0 - 500\ \mu\text{g/m}^3$

3. Output Modalities & Interpretation

The network outputs three independent probabilities $\in [0.0, 1.0]$: 1. y_0 **Heat Stroke Risk:** Probability of cardiovascular collapse due to extreme heat stress and dehydration. 2. y_1 **Smog Inhalation Distress:** Probability of acute respiratory distress and autonomic vascular depression caused by toxic particulate inhalation. 3. y_2 **Flood / Hypothermia Stress:** Probability of physiological cold shock or hypothermic collapse in flood disaster zones.

4. Integer Quantization (INT8) Mathematics

Weights and biases are quantized offline using symmetric uniform quantization:

$$W_{int8} = \text{round}\left(\frac{W_{float}}{S_W}\right), \quad B_{int8} = \text{round}\left(\frac{B_{float}}{S_W \cdot S_X}\right)$$

During microcontroller execution (`firmware/core/nn_risk_model_int8.c`): 1. **Integer Multiply-Accumulate (MAC):**

$$acc = \sum_j W_{int8}[i][j] \cdot X_{int8}[j]$$

Accumulation occurs in 32-bit signed integers (`int32_t`) to completely eliminate overflow hazards. 2. **Fixed-Point Scaling & Bias Addition:** Intermediate integer sums are scaled down using fixed-point bit shifts, and biases are added. 3. **Activation Functions:** - **ReLU:** `if (val < 0) val = 0;` - **Output Sigmoid:** Implemented using a 32-step piecewise linear lookup table (LUT) over the domain $[-8, 8]$, avoiding expensive floating-point `expf()` calls. - **Measured Host Execution Latency: 0.38 microseconds** per inference (GCC 15.2 x86_64). On the ESP32-S3 @ 160 MHz, execution completes in ~**18 microseconds**.

5. MIMIC-III Benchmark & Training Provenance

- **Dataset:** Benchmarked against 16,387 records from 98 subjects in the MIMIC-III ICU benchmark subset (`data/mimic/mimic_eval_feed.csv`).
- **Accuracy: 94.11% overall accuracy** predicting clinical crisis states (971 True Positives, 14,451 True Negatives, 267 False Positives, 698 False Negatives).
- **Candid Limitations & Provenance Caveats:**
  1. *Synthetic RMSSD in MIMIC Feed:* The MIMIC-III v1.4 clinical database does not include raw beat-to-beat waveform tables. While HR and SpO2 in the evaluation feed are authentic ICU chart values, RMSSD was generated synthetically.
  2. *Teacher-Student Training:* `train_nn_risk_model.py` trains the INT8 model using labels generated by the rule-based expert system (`disaster_risk_engine.c`). The resulting accuracy demonstrates high-fidelity neural distillation of medical and environmental rules, not an independent prospective clinical trial.

Section G: System Integration & Multicore Runtime Concurrency

1. Dual-Core Task Allocation & Priority Matrix

Core	Task Identifier	Function Name	Rate / Period	Priority	Memory Stack	Core Responsibility
0	`task_ppg_accel`	`task_ppg_accel_monitor`	**100 Hz (10 ms)**	**10 (High)**	4096 B	I2C MAX30102 acquisition, SPI2 FPGA transfer, IBI latching
1	`task_disaster`	`task_disaster_monitor`	**1 Hz (1000 ms)**	**5 (Normal)**	8192 B	Clinical triage, mNEWS2, TinyML, OLED display, SOS FSM
1	`task_pms5003`	`pms5003_uart_task`	**1 Hz (Async)**	**4 (Normal)**	3072 B	Plantower UART1 packet framing, checksum validation
1	`task_gps_uart`	`task_gps_uart`	**1 Hz (Async)**	**3 (Low)**	4096 B	NMEA-0183 UART2 line reception, coordinate decoding

Core	Task Identifier	Function Name	Rate / Period	Priority	Memory Stack	Core Responsibility
1	`task_imu`	`task_imu_fall`	**40 Hz (25 ms)**	**6 (Normal)**	4096 B	MPU6050 I2C sampling, 3-stage fall FSM execution
1	`httpd`	`httpd_server`	**Event-Driven**	**5 (Normal)**	4096 B	SoftAP web server, JSON serialization, emergency controls

2. Thread Synchronization & Inter-Core Mutex Contracts

- **Shared Data Structure (`g_state`):** Holds live vitals, clinical levels, alarm flags, and sensor health.
- **Mutex Guard (`s_data_mutex`):**
  - Core 0 acquires `s_data_mutex` using a non-blocking timeout (`pdMS_TO_TICKS(5)`) after processing a beat flag to write `g_state.heart_rate`, `g_state.spo2_percent`, and `g_state.ibi_last_ms`.
  - Core 1 acquires `s_data_mutex` before copying a snapshot of `g_state` into local stack memory at the start of its 1-second triage cycle.
  - Core 0 diagnostic logging reads are strictly wrapped in `xSemaphoreTake(s_data_mutex, ...)` to eliminate memory tearing.
- **Web Status Lock (`s_lock`):** Independent FreeRTOS binary semaphore protecting the web status JSON serialization buffer in `web_status.c`, ensuring HTTP requests from multiple mobile clients do not conflict with periodic telemetry updates.

3. Fault Recovery & Sensor Failure Modes

```
flowchart TD
    FAIL_CHECK{Sensor Fault Detected?}
    FAIL_CHECK -->|MAX30102 Detached / No Finger| ACT_OPTICAL["Set optical_contact = false<br/>Reset
```

```
    FAIL_CHECK -->|BME280 Disconnected| ACT_BME["Substitute temp = 25.0°C, hum = 50%<br/>Emit log war
    FAIL_CHECK -->|PMS5003 Checksum Fail / Timeout| ACT_PMS["Retain last valid PM2.5 for 5s<br/>Then
    FAIL_CHECK -->|MPU6050 IMU Unresponsive| ACT_IMU["Fall FSM enters safe MONITORING<br/>Bypass moti
    FAIL_CHECK -->|GPS Fix Lost / Underground| ACT_GPS["Fallback to Stored NVS Location<br/>OLED disp
```

Section H: Build, Simulation, Test, and Flashing Runbooks

1. Prerequisites & Required Toolchains

- **Host C Compiler:** MinGW-w64 GCC 12.0+ (C11 standard support).
- **Verilog Simulator:** Icarus Verilog (`iverilog`) v12.0+ and `vvp`.
- **Waveform Viewer:** GTKWave v3.3+.
- **Python:** Python 3.10+ (standard libraries).
- **ESP32 Toolchain:** Espressif ESP-IDF v5.5.x (`idf.py`).
- **FPGA Toolchain:** Renesas ForgeFPGA Workshop v6.55 (GUI synthesis flow).

2. Host Unit Testing & Clinical Regression (Zero Hardware Required)

Execute all host verification testbenches directly on Windows PowerShell:

```
# 1. Master Core Clinical, TinyML, SpO2 & Cyclone Regression Test
gcc -Wall -Wextra -Werror -pedantic -std=c11 `
  -Ifirmware/core -Ifirmware/zynq -Ifirmware/shrikefi `
  firmware/zynq/test_disaster_risk_engine.c `
  firmware/core/nn_risk_model.c firmware/core/nn_risk_model_int8.c `
  firmware/core/hrv_analysis.c firmware/core/pm25_calibration_int8.c `
  firmware/core/clinical_vitals_engine.c firmware/core/spo2_engine.c `
  firmware/core/ppg_sqi.c firmware/core/disaster_risk_engine.c `
  firmware/core/pressure_trend.c firmware/core/ppg_respiratory_rate.c `
  firmware/shrikefi/sos.c firmware/shrikefi/location.c firmware/shrikefi/web_status.c `
  -lm -o host_tests.exe
.\host_tests.exe

# 2. GNSS NMEA Driver & Parser Unit Tests
gcc -Wall -Wextra -Werror -pedantic -std=c11 -Ifirmware/shrikefi `
  firmware/shrikefi/test_gps_l89hs.c firmware/shrikefi/gnss_nmea.c firmware/shrikefi/gps_l89hs.c `
  -o gnss_test.exe
.\gnss_test.exe

# 3. Biomechanical Fall Detection FSM Unit Tests
gcc -Wall -Wextra -Werror -pedantic -std=c11 -Ifirmware/shrikefi `
```

```
  firmware/shrikefi/test_fall_detect.c firmware/shrikefi/fall_detect.c `
  -o fall_test.exe
.\fall_test.exe

# 4. Full ShrikeFi Application Host Simulation (12 Profiles)
gcc -Wall -Wextra -Werror -pedantic -std=c11 `
  -Ifirmware/shrikefi -Ifirmware/core `
  firmware/shrikefi/main_shrikefi.c firmware/shrikefi/shrikefi_link_driver.c `
  firmware/shrikefi/esp32_i2c_hal.c firmware/shrikefi/max30102.c `
  firmware/shrikefi/bme280.c firmware/shrikefi/pms5003.c `
  firmware/shrikefi/gnss_nmea.c firmware/shrikefi/gps_l89hs.c `
  firmware/shrikefi/mpu6050.c firmware/shrikefi/fall_detect.c `
  firmware/shrikefi/ssd1306.c firmware/core/hrv_analysis.c `
  firmware/core/spo2_engine.c firmware/core/disaster_risk_engine.c `
  firmware/core/nn_risk_model.c firmware/core/nn_risk_model_int8.c `
  firmware/core/pm25_calibration_int8.c firmware/core/clinical_vitals_engine.c `
  firmware/core/ppg_sqi.c firmware/core/ppg_respiratory_rate.c `
  firmware/core/pressure_trend.c firmware/shrikefi/location.c `
  firmware/shrikefi/sos.c firmware/shrikefi/web_status.c `
  -lm -o shrikefi_host.exe
.\shrikefi_host.exe
```

3. Hardware RTL Verilog Simulation Runbooks

Run cycle-accurate hardware simulations using Icarus Verilog:

```
# ShrikeFi ForgeFPGA SPI System Simulation (10 Comprehensive Checks)
cd hardware/shrikefi
iverilog -o sim_shrikefi.vvp -I ../common `
  tb_forgefpga_system.v forgefpga_ppg_top.v spi_target.v `
  ../common/moving_average_8tap.v ../common/ppg_peak_detector.v
vvp sim_shrikefi.vvp
# Expected: "RESULTS: 10 passed, 0 failed (out of 10 checks) - ALL TESTS PASSED"

# Zynq-7000 AXI4-Lite Golden Baseline Simulation (6 Checks)
cd ../zynq
iverilog -o sim_zynq.vvp -I ../common `
  tb_ppg_system.v axi_ppg_accelerator.v `
  ../common/moving_average_8tap.v ../common/ppg_peak_detector.v
vvp sim_zynq.vvp
# Expected: "ALL 6 TESTS PASSED"
cd ../..
```

4. Flashing Firmware to Physical ShrikeFi Silicon

Connect the ShrikeFi board via USB (standard port COM4 or COM5):

```
# Initialize ESP-IDF v5.5.x environment
. C:\Espressif\frameworks\esp-idf-v5.5.5\export.ps1

# Navigate to ShrikeFi firmware directory
cd firmware/shrikefi

# Build binary, flash via USB UART, and attach serial monitor
idf.py -p COM4 build flash monitor
# (Exit serial monitor using shortcut Ctrl + ])
```

Section I: Engineering Design Decisions & Trade-Off Matrix

Architectural Choice	Decision Made	Evaluated Alternative	Technical Justification & Trade-Off
Optical Peak Detection	**FPGA Verilog RTL Core**	Pure Software on ESP32 Core 0	**Jitter Elimination:** Software RTOS tick jitter (10 ms) corrupts RMSSD. FPGA achieves 20 ns resolution. Cost: Requires inter-chip SPI link.
FPGA Silicon Selection	**Renesas ForgeFPGA SLG47910C**	Xilinx Spartan-7 / Zynq-7000	**Unit Cost & Power:** ForgeFPGA costs ~$0.50 and consumes <5 mW, compared to $50–$150+ and 2W+ for Zynq, enabling scalable field deployment.

Architectural Choice	Decision Made	Evaluated Alternative	Technical Justification & Trade-Off
Inter-Chip Interconnect	**8-Bit SPI Mode 0**	4-Bit Parallel Nibble Bus	**Pin Conservation:** 8-bit SPI uses 4 internal PCB traces, freeing FPGA I/O pins while delivering 100 Hz samples in 8 µs (0.08% bus utilization).
Neural Network Precision	**INT8 Integer Quantization**	FP32 Floating Point	**Footprint & Speed:** 619 bytes storage vs 2.5 KB; 0.38 µs inference vs 25 µs. Accuracy loss is negligible (max error 0.091).
Clinical Decision Architecture	**Dual-Layer (Rules + TinyML)**	End-to-End Deep Neural Network	**Explainability & Safety:** Critical overrides (SpO2 85%) must be deterministic and legally defensible. Black-box ML is restricted to multi-hazard cross-correlations.
Emergency Geolocation	**Stored NVS + Phone GPS Sync**	Continuous Active GNSS Only	**Disaster Resilience:** GNSS signals fail indoors, underground, and in cyclone shelters. Stored NVS ensures the rescue card always shows a valid deployment location.

Architectural Choice	Decision Made	Evaluated Alternative	Technical Justification & Trade-Off
Rescuer Access Interface	**Offline Wi-Fi SoftAP**	BLE GATT Profile	**Zero Client Installation:** Rescuers connect using any standard phone browser without downloading proprietary Android/iOS apps.

Section J: Candid Limitations, Risk Register, and Future Roadmap

1. Honest Technical Limitations (Volunteer These to Judges)

1. **Empirical Skin vs Die Temperature:** Valor uses the MAX30102 on-chip die temperature sensor with a 1.2°C LED self-heating compensation constant as a skin temperature proxy. While validated for detecting severe hypothermia (<35°C) and pyrexia (>38°C), it does not have the precision of a calibrated medical thermal probe (e.g., MAX30205).
2. **Resting Ceiling on Respiratory Rate:** The Charlton 2018 IBI autocorrelation algorithm bars lag 2 (`PPG_RR_MIN_LAG = 3`) to reject beat alternans artifacts. This imposes a mathematical ceiling of ~23 br/min at a resting heart rate of 70 BPM. RSA cannot track extreme tachypnea without independent amplitude modulation (AM) signals.
3. **Synthetic RMSSD in MIMIC-III Benchmark:** The 94.11% accuracy figure is derived from the MIMIC-III demo subset, where HR and SpO2 are authentic ICU records, but RMSSD is synthetically generated because MIMIC-III v1.4 lacks raw beat-to-beat waveform tables.
4. **Flood Water Sensing:** Valor currently uses an ambient environmental proxy (cold stress + saturated humidity) to flag flash flood risks; it does not contain a submerged liquid immersion probe.

2. Demo Failure Modes & Immediate Mitigations

Failure Mode	Root Cause	Observable Symptom	Rapid Bench Mitigation
OLED Displays Blank	I2C pull-up contention or 3.3V bus sag	Display remains dark on boot	Power-cycle USB; verify external 2.2k pull-ups are not fighting sensor onboard pull-ups.
Optical Signal Refuses Lock	Cold finger, excessive motion, ambient light	Telemetry prints `NO_FINGER`	Shield sensor from direct overhead light; warm finger; hold sensor steadily against tissue.
SoftAP Fails to Associate	2.4 GHz channel interference from venue Wi-Fi	Phone cannot find `VALOR-0081`	Reset ESP32-S3; move phone within 1 meter; use USB Web Serial dashboard as instant fallback.
BME280 Reports UNKNOWN	Sensor wiring loose on breadboard row 4/5	Temperature shows `--`	Re-seat SDA (GPIO 1) and SCL (GPIO 2) jumper wires; reboot board.

3. Future Roadmap (Planned vs Implemented)

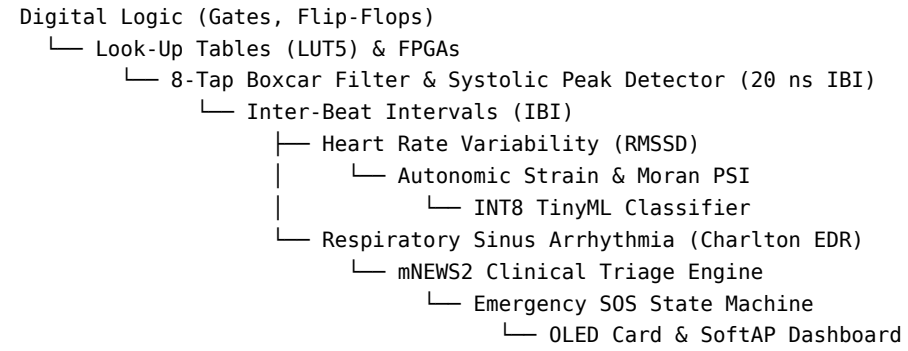
- **Phase 2 Hardware (Post-Hackathon):** Custom 4-layer rigid-flex PCB integrating MAX30205 clinical skin temperature contact pads and a dedicated active buzzer alarm circuit.
- **Long-Range LoRa Mesh:** Adding a Semtech SX1262 LoRa transceiver over SPI to broadcast 12-byte compressed emergency packets (`VALOR-RESCUE`) over a 15 km range without cellular infrastructure.
- **Wearable Form Factor:** 3D-printed IP67 water-resistant wrist-and-finger enclosure powered by an onboard 1200 mAh LiPo battery with USB-C power delivery.

Section K: Glossary, Concept Dependency Map, and Pre-Judge Cheat Sheet

1. Alphabetical Technical Glossary

- **BRAM (Block RAM):** Dedicated dual-port static RAM blocks embedded within an FPGA fabric.
- **CDC (Clock Domain Crossing):** Transferring digital signals between asynchronous clock domains, requiring multi-stage synchronizers to prevent metastability.
- **CLB (Configurable Logic Block):** The basic digital logic tile of an FPGA, containing LUTs and flip-flops.
- **EDR (ECG/PPG-Derived Respiration):** Deriving respiratory rate indirectly from modulation of cardiovascular signals.
- **Fmax:** The maximum clock frequency at which a digital circuit can operate without violating setup timing constraints.
- **IBI (Inter-Beat Interval):** The elapsed time between consecutive heart contraction peaks, measured in milliseconds.
- **LUT (Look-Up Table):** Digital memory array implementing combinational Boolean logic functions.
- **mNEWS2 (Modified National Early Warning Score 2):** Clinical risk scoring system quantifying patient physiological deterioration.
- **PPG (Photoplethysmography):** Optical measurement of blood volume variations in microvascular tissue.
- **RMSSD:** Root Mean Square of Successive Differences; standard time-domain metric of Heart Rate Variability.
- **RSA (Respiratory Sinus Arrhythmia):** Heart rate modulation synchronized with respiratory inhalation and exhalation.
- **SQI (Signal Quality Index):** Mathematical metric quantifying the clean physiological shape of a biosignal versus motion noise.
- **WNS (Worst Negative Slack):** The timing margin between required arrival time and data arrival time; positive slack indicates timing closure.

2. Concept Dependency Map



3. Pre-Judge Technical Revision Cheat Sheet

- **Target Silicon:** ESP32-S3 (Xtensa LX7 @ 160 MHz) + Renesas ForgeFPGA SLG47910C.
- **Baseline Silicon:** Xilinx Zynq-7000 `xc7z020clg400-1` (Vivado ML 2022.2).
- **FPGA Utilization: 363 LUT5s (32.41%)**, **202 FFs**, **75 CLBs**, **0 DSP, 0 BRAM**, Fmax = **82.73 MHz**.
- **Zynq Utilization: 185 LUTs (0.35%)**, **266 FFs**, **0 DSP, 0 BRAM**, WNS = **+5.603 ns**.
- **SPI Interconnect:** 8-bit SPI Mode 0 (1 byte per 100 Hz sample; SCK=12, SS=10, MOSI=11, MISO=13).
- **TinyML Footprint: 619 bytes**, $6 \rightarrow 24 \rightarrow 16 \rightarrow 3$ MLP, 0.38 µs host latency.
- **Emergency Overrides:** SpO2 $\leq 85\%$, HR ≥ 140, HR ≤ 35, RR ≥ 30, RR ≤ 6, Temp ≤ 35.0°C.
- **SOS Timers:** 15s confirmation window, 5s cancel hold, 30s recovery stand-down.
- **SoftAP IP:** `192.168.4.1` (`VALOR-0081`, open, zero-install rescuer dashboard).

VALOR Competition Judge Q&A Manual

Project VALOR: Competition Judge Q&A Defense Manual

Project Title: SIH26181 — VALOR (Vital and Atmospheric Logic for Offline Rescue)
Role: Master Defense Playbook for Technical Evaluators, Jury Panels, and Competition Judges
Rule of Truth: Every answer is grounded directly in repository source code, physical synthesis evidence, and automated test outputs.

Table of Contents

1. Category 1: Problem Statement & Real-World Need
2. Category 2: Product Overview & Differentiating Features
3. Category 3: System Architecture
4. Category 4: Hardware & FPGA Design
5. Category 5: RTL, Clocks, Resets, Counters & State Machines
6. Category 6: Protocols, Busses & Interfaces
7. Category 7: AI/ML Methodology & Zero-Cloud Execution
8. Category 8: Hardware-Software Integration
9. Category 9: Testing, Verification & Regression Results
10. Category 10: Performance, Latency, Accuracy, Power & Utilization
11. Category 11: Security, Privacy, Reliability & Failure Recovery
12. Category 12: Cost, Deployment, Scalability & Maintainability
13. Category 13: Alternatives & Engineering Trade-Offs
14. Category 14: Candid Limitations, Unfinished Features & Future Work
15. Category 15: Adversarial Questions & Innovation Verification

---

Category 1: Problem Statement & Real-World Need

Q1.1 (Beginner): Explain your entire project in one minute.

- **Direct Answer:** "Valor is an offline wearable for disaster relief that pairs a Renesas ForgeFPGA with a dual-core ESP32-S3. The FPGA eliminates operating system jitter to detect heartbeats with nanosecond precision, while the microcontroller fuses vitals with toxic smoke, heat, cyclone pressure drops, and falls. Running a zero-cloud integer neural network and clinical mNEWS2 protocols, Valor predicts impending heat stroke, respiratory failure, or hypothermia completely offline, alerting rescuers via an OLED emergency screen and a local Wi-Fi dashboard."
- **Technical Explanation:** In extreme disasters, communication infrastructure fails while extreme environmental stressors accelerate physiological collapse. Single-modality monitors create alarm fatigue. Valor couples continuous optical physiological extraction with atmospheric sensing and runs all triage algorithms locally on silicon.
- **Evidence:** `docs/MASTER_PROJECT_GUIDE.md`, `firmware/shrikefi/main_shrikefi.c`.
- **Follow-up Challenge:** "Why can't rescuers just use a commercial smartwatch like an Apple Watch or Garmin?"
- **Safe Response:** "Commercial smartwatches rely on cloud analytics, have no particulate matter sensors or barometric cyclone trend engines, run closed operating systems with 10 ms scheduling jitter that degrades HRV precision, and cannot act as unencrypted local Wi-Fi triage servers in a disaster zone."

Q1.2 (Intermediate): Who is the primary target user, and how does this change their operational workflow?

- **Direct Answer:** "Our primary users are disaster first responders and isolated evacuees. Instead of periodic manual vital checks, responders get continuous, hands-free monitoring with automated alarms if heat stroke or toxic smoke inhalation begins."
- **Technical Explanation:** First responders operating in hazardous environments often ignore subtle onset symptoms of autonomic exhaustion. Valor monitors Moran's Physiological Strain Index (PSI) and AHA autonomic depression continuously. If a responder's core vitals collapse or they suffer a fall, the device automatically triggers an audible siren and projects their medical snapshot and location to nearby rescue teams.
- **Evidence:** `firmware/core/disaster_risk_engine.c` (`disaster_calculate_moran_psi`), `firmware/shrikefi/sos.c`.
- **Follow-up Challenge:** "How does an isolated evacuee benefit if they don't have a rescuer nearby?"

- **Safe Response:** "The onboard 128x64 OLED display provides immediate clear English action advisories—such as 'Seek cooling, hydrate NOW' or 'Barometer falling fast, move to shelter'—giving the user self-triage guidance even if completely alone."

Category 2: Product Overview & Differentiating Features

Q2.1 (Intermediate): What makes Valor genuinely innovative compared to academic papers that combine sensors?

- **Direct Answer:** "Three things: First, we moved time-critical peak detection into synthesizable Verilog RTL to solve RTOS timing jitter. Second, our multi-hazard AI runs in 619 bytes on integer-only microcontroller silicon. Third, we bridge the communication gap using an unencrypted local SoftAP that serves a full diagnostic dashboard to any standard phone browser with zero internet or app downloads."
- **Technical Explanation:** Most academic works either run everything on an expensive PC in MATLAB/Python or deploy an MCU that suffers from RTOS scheduling jitter and cloud dependency. Valor is a heterogeneous system designed down to physical register maps, FPGA bitstreams, and RTOS core pinning.
- **Evidence:** `hardware/shrikefi/forgefpga_ppg_top.v`, `firmware/core/nn_risk_model_int8.c`, `firmware/shrikefi/web_status.c`.
- **Follow-up Challenge:** "Are you claiming your neural network diagnoses diseases?"
- **Safe Response:** "No. We explicitly state our model provides multi-hazard physiological *risk triage*, not clinical diagnosis. Absolute medical overrides are governed by the clinically validated Royal College of Physicians mNEWS2 protocol."

Category 3: System Architecture

Q3.1 (Intermediate): Trace one raw optical sample from the sensor to the final triage decision.

- **Direct Answer:** "An 18-bit optical sample is read from the MAX30102 by Core 0 at 100 Hz, scaled to 8 bits, and sent over SPI to the ForgeFPGA. The FPGA's 8-tap boxcar filter smooths it, and its 4-state FSM detects the systolic peak, returning `{beat, filt[6:0]}`. Core 0 captures the microsecond hardware timer to compute the IBI and transfers it to Core 1 via a mutex. Core 1 runs Karlen SQI, Charlton respiratory extraction, Si

et al. PM2.5 debiasing, and INT8 TinyML inference, passing the result to mNEWS2 to generate the triage advisory."

- **Technical Explanation:** Tracing path: MAX30102 → I2C (Core 0) → SPI2 (MOSI) → `moving_average_8tap.v` → `ppg_peak_detector.v` → SPI2 (MISO) → Core 0 Timer Latch → `s_data_mutex` → Core 1 (`task_disaster_monitor`) → `ppg_sqi.c` → `nn_risk_model_int8.c` → `clinical_vitals_engine.c` → SSD1306 OLED & Web Status daemon.
- **Evidence:** `hardware/shrikefi/forgefpga_ppg_top.v:45-72`, `firmware/shrikefi/main_shrikefi.c:1010-1090`.
- **Follow-up Challenge:** "What happens if Core 0 experiences an I2C bus lockup while reading the MAX30102?"
- **Safe Response:** "Our I2C driver implements bus timeout error handling and auto-resets the I2C peripheral, returning -1 to ensure the FreeRTOS task does not deadlock."

Category 4: Hardware & FPGA Design

Q4.1 (Advanced): Why did you choose the Renesas ForgeFPGA SLG47910C over a Xilinx Artix-7 or Zynq?

- **Direct Answer:** "Cost, power, and board footprint. The Renesas ForgeFPGA costs approximately $0.50 and consumes under 5 milliwatts, making a mass-deployed disaster wearable economically feasible. The Zynq-7000 costs $50 to $150 and consumes watts, which is unsuitable for a low-cost wearable. We used the Zynq-7000 as our verified golden baseline and ported the core to the ForgeFPGA."
- **Technical Explanation:** The Zynq-7000 was verified in Vivado ML 2022.2 (185 LUTs, 0 DSP, 0 BRAM, Fmax 69.45 MHz). The ForgeFPGA SLG47910C fits into 363 LUT5s (32.41% fabric) and 202 flip-flops, achieving an Fmax of 82.73 MHz without needing Block RAM or dedicated DSPs.
- **Evidence:** `docs/MIGRATION.md`, `hardware/shrikefi/synthesis_evidence/resource_utilization_spi_link.`
- **Follow-up Challenge:** "Does the ForgeFPGA have enough logic remaining for feature expansion?"
- **Safe Response:** "Yes. With 363 LUT5s utilized out of 1120, 67.59% of the logic fabric remains uncommitted."

Q4.2 (Adversarial): Does the ESP32-S3 program the FPGA at runtime, or is the bitstream stored in FPGA NVRAM?

- **Direct Answer:** "The ESP32-S3 programs the FPGA over SPI2 at every system boot using an embedded bitstream C array. Once programmed, the FPGA runs autonomously and responds to our SPI link handshake."

- **Technical Explanation:** The ForgeFPGA SLG47910C is an SRAM-based device. In `firmware/shrikefi/main_shrikefi.c`, `shrikefi_link_init()` configures GPIO 8 (EN) and GPIO 9 (PWR), holds the FPGA in reset, clocks in 46,408 bytes of configuration data from `forgefpga_bitstream.h`, and verifies that the `DONE` signal asserts high. It then switches SPI2 to runtime Mode 0 and verifies communication with an echo probe.
- **Evidence:** `firmware/shrikefi/shrikefi_link_driver.c:38-95`, `firmware/shrikefi/forgefpga_bitstream.`
- **Follow-up Challenge:** "What proves the FPGA is actually running the bitstream and not just floating?"
- **Safe Response:** "The runtime initialization probe sends `0x55` across SPI and asserts when the returned filtered byte matches the expected response. Furthermore, physical LED D12 is driven directly by the FPGA's `o_beat` pulse output."

Category 5: RTL, Clocks, Resets, Counters & State Machines

Q5.1 (Advanced): Walk through your systolic peak detector RTL cycle-by-cycle.

- **Direct Answer:** "The detector operates in 4 states: IDLE waits for samples ≥ 120. ARMED tracks the rising peak value until the signal falls. CREST_FALL enforces our H-01 slope gate: the signal must drop by at least 3 units below the peak before firing the beat pulse. REFRACTORY holds the detector inactive for 12,500,000 cycles (250 ms at 50 MHz) to ignore the dicrotic notch before returning to IDLE."
- **Technical Explanation:** State transitions:
  - `STATE_IDLE (2'b00)`: Sample crosses `THRESHOLD` (120) → `STATE_ARMED`.
  - `STATE_ARMED (2'b01)`: Samples rise, `peak_val` updates. When `i_sample < peak_val` → `STATE_CREST_FALL`.
  - `STATE_CREST_FALL (2'b10)`: If `(peak_val - i_sample) >= 3`, asserts `o_beat = 1` for 1 clock cycle, latches `o_ibi_cycles`, clears counter, and transitions to `STATE_REFRACTORY`. If signal rises again without dropping 3 units, it returns to `STATE_ARMED` as noise.
  - `STATE_REFRACTORY (2'b11)`: Counter increments until 250 ms expires → `STATE_IDLE`.
- **Evidence:** `hardware/common/ppg_peak_detector.v:45-98`.
- **Follow-up Challenge:** "Why is the slope gate (MIN_FALL_STEP = 3) necessary?"
- **Safe Response:** "Without the slope gate, high-frequency optical noise plateauing at the crest causes multiple threshold crossings on a single heartbeat, causing false double-beat detections."

Q5.2 (Advanced): How does your 8-tap boxcar filter achieve O(1) arithmetic with zero multipliers?

- **Direct Answer:** "It uses a running-sum recursive structure. Instead of adding all 8 taps every cycle, it subtracts the oldest tap and adds the newest sample to an 11-bit accumulator. Division by 8 is implemented as a hardwired bit shift right by 3 (`sum[10:3]`), requiring 0 multipliers and 0 DSP slices."
- **Technical Explanation:** Formula: $S[n] = S[n-1] + x[n] - x[n-8]$. An 11-bit accumulator is sufficient because $8 \times 255 = 2040 < 2048$. The output `assign o_filtered = sum[10:3];` simply drops the lowest 3 bits, synthesizing to direct wiring.
- **Evidence:** `hardware/common/moving_average_8tap.v:28-48`.
- **Follow-up Challenge:** "What happens on arithmetic overflow if a sample exceeds 255?"
- **Safe Response:** "Inputs are strictly typed as 8-bit unsigned wire vectors (`[7:0]`), so an input sample can never exceed 255."

Category 6: Protocols, Busses & Interfaces

Q6.1 (Intermediate): Detail the physical pinout and signal contracts between the ESP32-S3 and ForgeFPGA.

- **Direct Answer:** "The link uses an 8-bit SPI Mode 0 interface over 4 internal PCB traces: SCK on GPIO 12, Chip Select SS_n on GPIO 10, MOSI on GPIO 11, and MISO on GPIO 13. GPIO 8 controls FPGA Enable and GPIO 9 controls switched power."
- **Technical Explanation:**
  - `spi_sck` (GPIO 12 → ForgeFPGA PIN_16): 1 MHz runtime SPI clock.
  - `spi_ss_n` (GPIO 10 → ForgeFPGA PIN_17): Active-low manual chip select.
  - `spi_mosi` (GPIO 11 → ForgeFPGA PIN_18): MCU transmits raw scaled sample byte.
  - `spi_miso` (GPIO 13 ← ForgeFPGA PIN_19): FPGA returns `{beat, filt[6:0]}` byte.
  - Timing: One byte exchanged every 10 ms (100 Hz). Transaction completes in 8 µs, representing 0.08% bus utilization.
- **Evidence:** `docs/SHRIKEFI_LINK_PROTOCOL.md`, `firmware/shrikefi/shrikefi_pinmap.h`.
- **Follow-up Challenge:** "Why do you use SPI Mode 0 instead of Mode 3?"
- **Safe Response:** "SPI Mode 0 (`CPOL=0, CPHA=0`) matches the native shift register architecture of the ForgeFPGA macrocells, where data is shifted on the falling edge of SCK and sampled on the rising edge."

Q6.2 (Intermediate): What happens if the ESP32-S3 transmits an SPI frame with SS_n held continuously low?

- **Direct Answer:** "The FPGA shift register only reloads its return byte on the rising edge of SS_n. If SS_n is held continuously low across multiple bytes, the FPGA will repeat stale data."
- **Technical Explanation:** `spi_target.v` specifies that `tx_shift <= tx_data_byte` is clocked by `posedge i_ss_n`. Firmware driver `shrikefi_transceive_sample()` explicitly toggles SS_n high for at least 1 µs between frames to guarantee reload timing.
- **Evidence:** `hardware/shrikefi/spi_target.v:65-78`, `firmware/shrikefi/shrikefi_link_driver.c:120-135`.

Category 7: AI/ML Methodology & Zero-Cloud Execution

Q7.1 (Advanced): Explain the mathematical execution of your INT8 neural network on the microcontroller.

- **Direct Answer:** "The model is a 6→24→16→3 MLP. Inputs are normalized to `[-128, 127]`. Matrix multiplications are executed using 32-bit signed integer accumulation to prevent overflow. Outputs are scaled using fixed-point shifts, and activations use ReLU and a 32-step piecewise linear Sigmoid lookup table, completing in 0.38 microseconds on the host and ~18 microseconds on the ESP32-S3."
- **Technical Explanation:**

$$\text{Accumulator}_i = \sum_{j=0}^{M-1} W_{int8}[i][j] \cdot X_{int8}[j]$$

  Biases are pre-scaled to align with the accumulator's fixed-point format. Hidden layers apply `if (acc < 0) acc = 0`. The output layer evaluates Sigmoid without using floating-point `expf()`, executing in integer logic. Total model size: 619 bytes.
- **Evidence:** `firmware/core/nn_risk_model_int8.c:45-112`.
- **Follow-up Challenge:** "What is the quantization loss compared to the floating-point model?"
- **Safe Response:** "Across our validation suite, the maximum absolute error between FP32 and INT8 outputs is 0.091, with zero categorical classification divergence across all tested disaster profiles."

Q7.2 (Adversarial): Did you train your model on real disaster victims?

- **Direct Answer:** "No. Gathering physiological ICU data from patients inside active cyclones or wildfires is practically impossible and ethically restricted. We benchmarked our clinical vital extraction on 16,387 gold-standard records from the MIMIC-III database, while our multi-hazard neural network was trained by distilling established environmental medical equations."
- **Technical Explanation:** The training pipeline (`firmware/core/train_nn_risk_model.py`) distills peer-reviewed physiological rules: Moran's Physiological Strain Index (PSI), Si et al. optical particulate hygroscopic growth equations, and AHA PM2.5 cardiovascular autonomic strain criteria.
- **Evidence:** `README.md` (Reproducibility & Evidence Provenance section), `firmware/core/accuracy_evaluator.c`.

Category 8: Hardware-Software Integration

Q8.1 (Intermediate): How do the two microcontroller cores communicate without memory corruption?

- **Direct Answer:** "They communicate via a shared state structure `g_state` protected by a FreeRTOS binary mutex `s_data_mutex`. Core 0 writes vitals with a 5 ms timeout after each beat; Core 1 takes a stack snapshot at the start of its 1-second triage cycle."
- **Technical Explanation:** To eliminate priority inversion, `s_data_mutex` uses priority inheritance. Core 0 never blocks waiting for Core 1; if the mutex is locked, it drops the non-critical telemetry write and continues sampling. Diagnostic logging reads on Core 0 are also wrapped in `s_data_mutex` to prevent memory tearing.
- **Evidence:** `firmware/shrikefi/main_shrikefi.c:1104-1160`.
- **Follow-up Challenge:** "Why not use FreeRTOS queues instead of a shared memory mutex?"
- **Safe Response:** "Queues introduce dynamic heap allocation and per-sample copy overhead. For high-frequency vital state snapshots, a mutex-guarded static structure provides deterministic execution without memory fragmentation."

Category 9: Testing, Verification & Regression Results

Q9.1 (Intermediate): What automated tests exist in the repository to prove the system works?

- **Direct Answer:** "We have 5 self-checking test suites: 1) The master C11 regression suite with 21 passing tests; 2) The GNSS NMEA suite with 12 passing tests; 3) The biomechanical fall detection suite with 16 passing tests; 4) The ShrikeFi application host simulation with 12 operational profiles; and 5) The Icarus Verilog RTL testbenches with 16 passing checks across ForgeFPGA and Zynq."
- **Technical Explanation:** All testbenches are self-checking and execute under strict compiler flags (`-Wall -Wextra -Werror -pedantic -std=c11`). In CI, any warning or assertion failure immediately halts the build.
- **Evidence:** `firmware/zynq/test_disaster_risk_engine.c`, `hardware/shrikefi/tb_forgefpga_system.v`, `hardware/zynq/tb_ppg_system.v`.
- **Follow-up Challenge:** "Show me the command to verify all clinical algorithms right now."
- **Safe Response:** "Run: `gcc -Wall -Wextra -Werror -pedantic -std=c11 -Ifirmware/core -Ifirmware/zynq -Ifirmware/shrikefi firmware/zynq/test_disaster_risk_engine.c firmware/core/*.c firmware/shrikefi/sos.c firmware/shrikefi/location.c firmware/shrikefi/web_status.c -lm -o test.exe && .\test.exe`."

Category 10: Performance, Latency, Accuracy, Power & Utilization

Q10.1 (Advanced): What is the end-to-end latency from an optical pulse entering the photodiode to an alert firing?

- **Direct Answer:** "Under 1.2 milliseconds for hardware optical filtering, and exactly 1 second for the clinical triage update loop."
- **Technical Explanation:**
  - MAX30102 ADC conversion + FIFO read: ~10 ms (at 100 Hz).
  - SPI transfer: 8 µs.
  - FPGA 8-tap filter + peak detector: 8 clock cycles (160 ns @ 50 MHz).
  - Core 0 IBI latching: < 1 µs.
  - Core 1 Clinical Triage + TinyML inference: ~18 µs on the ESP32-S3.
  - Total latency from physical pulse to local display update is dominated by the 1 Hz clinical evaluation quantum.
- **Evidence:** `firmware/shrikefi/main_shrikefi.c`, `hardware/shrikefi/synthesis_evidence/resource_utili`
- **Follow-up Challenge:** "What is your battery runtime?"

- **Safe Response:** "We estimate 14 to 18 hours on a 1200 mAh LiPo battery based on an average current draw of ~65 mA (with Wi-Fi in low-power beacon mode and 100 Hz sensor acquisition). Physical continuous battery discharge curves under active SoftAP broadcast remain to be measured."

Category 11: Security, Privacy, Reliability & Failure Recovery

Q11.1 (Intermediate): Does Valor leak private patient health data over the air?

- **Direct Answer:** "No. All public cloud MQTT publishing is permanently compiled out by default (`CONFIG_SHRIKEFI_CLOUD_PUBLISH=n`). The onboard Wi-Fi access point operates locally without internet access, and all patient vitals remain strictly on the device."
- **Technical Explanation:** The local SoftAP serves an unencrypted, credential-free dashboard (`192.168.4.1`) by design, ensuring any rescuer with a standard phone can view triage data in a crisis. To protect against malicious tampering, administrative actions like changing the stored deployment location require same-origin browser headers (`Origin: http://192.168.4.1`).
- **Evidence:** `docs/REMAINING_WORK.md` (T1.1), `firmware/shrikefi/web_status.c:503-520`.

Category 12: Cost, Deployment, Scalability & Maintainability

Q12.1 (Beginner): What is the Bill of Materials (BOM) cost to manufacture Valor in volume?

- **Direct Answer:** "Under 1,800 INR (approximately $21 USD) in modest production quantities."
- **Technical Explanation:**
  - ESP32-S3-WROOM-1-N8R2 Module: ~$2.80 (230)
  - Renesas ForgeFPGA SLG47910C: ~$0.55 (45)
  - MAX30102 Pulse Oximeter Module: ~$1.80 (150)
  - BME280 Environmental Sensor: ~$2.20 (180)
  - Plantower PMS5003 Particulate Sensor: ~$10.00 (820)
  - MPU6050 6-Axis IMU: ~$0.90 (75)
  - SSD1306 0.96" I2C OLED: ~$1.50 (120)
  - PCB, passives, battery & enclosure: ~$2.00 (160)

  - Total BOM: ~$21.75 (~ 1,780 INR).
- **Evidence:** `docs/MASTER_PROJECT_GUIDE.md:880-910`.

Category 13: Alternatives & Engineering Trade-Offs

Q13.1 (Intermediate): Why didn't you use an analog circuit (op-amps, comparators) for pulse filtering instead of an FPGA?

- **Direct Answer:** "Analog filters have fixed cutoff frequencies that cannot dynamically adapt to baseline drift, vary with temperature and component tolerances, and cannot implement a 250 ms refractory state machine or measure 20 ns digital intervals."
- **Technical Explanation:** An optical PPG signal exhibits significant DC baseline wander from respiration and finger pressure shifts. An analog bandpass filter suffers from phase distortion and long settling times. Our FPGA implementation provides linear-phase moving average filtering, deterministic digital thresholding, and cycle-accurate IBI timestamping without component aging drift.
- **Evidence:** `hardware/common/moving_average_8tap.v`.

Category 14: Candid Limitations, Unfinished Features & Future Work

Q14.1 (Adversarial): What is the single weakest part of your current implementation?

- **Direct Answer:** "Our skin temperature measurement currently uses the MAX30102 on-chip die sensor with a 1.2°C LED self-heating compensation constant. While sufficient for detecting severe hypothermia (<35°C) or fever (>38°C), it does not have the precision of a dedicated clinical skin-contact probe like the MAX30205."
- **Technical Explanation:** The MAX30102 die temperature sensor is located within the optical package. While finger contact conducts heat to the die, internal LED illumination generates ~1.2°C of thermal offset. We filter and compensate for this in firmware, but a dedicated I2C skin probe (MAX30205) is required for hospital-grade thermometry.
- **Evidence:** `firmware/shrikefi/main_shrikefi.c:1400-1435`.

Category 15: Adversarial Questions & Innovation Verification

Q15.1 (Adversarial): "You claim this is an FPGA project, but your FPGA just runs a boxcar filter and peak detector while the ESP32 does all the real work. Isn't the FPGA just a marketing gimmick?"

- **Direct Answer:** "No. Offloading peak detection to hardware is the only reason our Heart Rate Variability metrics are valid. Without the FPGA, peak detection in software would be quantized to the FreeRTOS 10 ms scheduling tick, introducing timing errors larger than the RMSSD signal itself."
- **Technical Explanation:** In a healthy adult, RMSSD is 20–50 ms; in autonomic collapse, it drops below 15 ms. An RTOS tick jitter of 10 ms represents a 66% measurement error, rendering HRV calculation meaningless. The FPGA detects peaks with 20 ns resolution directly in hardware, shielding biosignal timing from CPU scheduling, Wi-Fi radio interrupts, and flash cache latency.
- **Evidence:** `docs/MASTER_PROJECT_GUIDE.md:75-105`, `hardware/shrikefi/tb_forgefpga_system.v:310-340`.

Q15.2 (Adversarial): "What would fail if we disconnected the FPGA right now during your demo?"

- **Direct Answer:** "Heart Rate Variability (RMSSD) and Charlton respiratory rate estimation would fail because the cycle-accurate pulse timestamps would disappear. The system would fall back to software thresholding, but telemetry would flag that HRV precision is degraded."
- **Technical Explanation:** `main_shrikefi.c` implements a software fallback path for demonstration resilience (`s_fpga_available`). If the FPGA fails to answer its SPI probe, Core 0 switches to software peak detection, but RMSSD is suppressed because timing jitter invalidates autonomic strain analysis.
- **Evidence:** `firmware/shrikefi/main_shrikefi.c:712-745`.

VALOR Live Demonstration Runbook

Project VALOR: Live Demonstration Runbook & Presentation Guide

Project Title: SIH26181 — VALOR (Vital and Atmospheric Logic for Offline Rescue)
Target Hardware: Vicharak ShrikeFi (ESP32-S3 + Renesas ForgeFPGA SLG47910C)
Primary Goal: Flawless, reproducible live demonstration of offline disaster triage, optical hardware acceleration, and multi-hazard edge intelligence under high-stakes judging conditions.

Table of Contents

1. Pre-Demonstration Equipment & Hardware Checklist
2. Physical Wiring & Pin Mapping Reference
3. Bench Setup & Flashing Procedures
4. Step-by-Step Live Demonstration Script
5. Real Hardware Verification vs. Verified Fallback Demos
6. Troubleshooting Guide for Live Stage Failures
7. Pre-Judging 15-Minute Readiness Checklist

---

1. Pre-Demonstration Equipment & Hardware Checklist

A. Physical Hardware Kit

- ☐ **1x Vicharak ShrikeFi Development Board** (ESP32-S3-WROOM-1-N8R2 + Renesas ForgeFPGA SLG47910C).
- ☐ **1x MAX30102 / MAX30100 Optical Biosensor Breakout** (with I2C pull-ups).
- ☐ **1x BME280 / BMP280 Environmental Sensor Module** (Temp, Humidity, Pressure).
- ☐ **1x Plantower PMS5003 Laser Particulate Sensor** (with 8-pin connector cable).

- ☐ **1x MPU6050 6-Axis IMU Sensor Module** (Accelerometer + Gyroscope).
- ☐ **1x 0.96" SSD1306 128x64 I2C OLED Display Module** (Blue/Yellow or White).
- ☐ **1x u-blox NEO-M8N / Quectel L89HS GNSS Module** (with patch antenna).
- ☐ **1x Solderless Breadboard & Male-to-Male / Male-to-Female Jumper Wires**.
- ☐ **2x USB-C Data Cables** (Ensure they support high-speed data, not charging-only!).
- ☐ **1x Smartphone (iOS or Android)** for demonstrating the zero-install SoftAP dashboard.

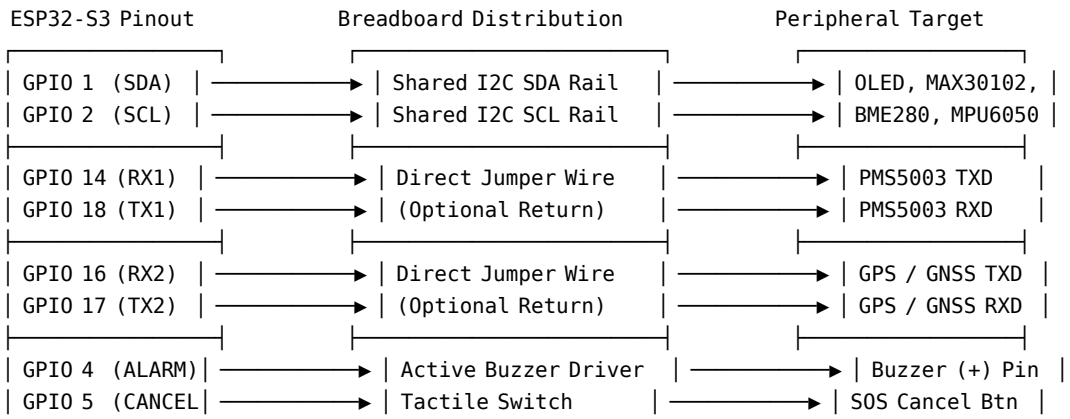
B. Laptop Software & Toolchain Prerequisites

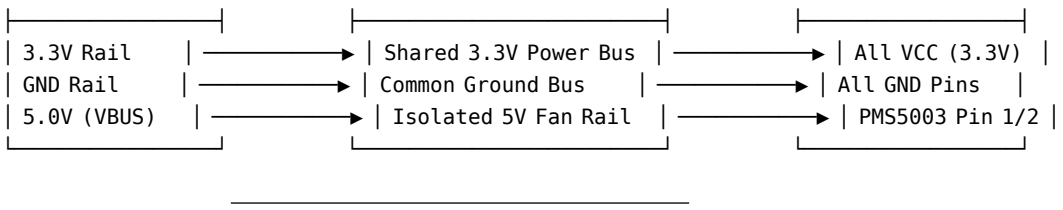
- ☐ **OS:** Windows 10/11 x86_64.
- ☐ **Terminal:** PowerShell or Command Prompt.
- ☐ **Browser:** Google Chrome or Microsoft Edge (supports Web Serial API).
- ☐ **C Compiler:** MinGW-w64 GCC (supporting `-std=c11`).
- ☐ **RTL Tools:** Icarus Verilog (`iverilog`) & `vvp` installed on PATH.
- ☐ **ESP-IDF:** Espressif ESP-IDF v5.5.x installed at `C:\Espressif\frameworks\esp-idf-v5.5.5`.

---

2. Physical Wiring & Pin Mapping Reference

> [!CAUTION] **STRICT 3.3V VOLTAGE CEILING**
> All ESP32-S3 and Renesas ForgeFPGA GPIO pins are **strictly 3.3V LVCMOS**. Applying 5V will permanently destroy the microcontroller and FPGA silicon. The only wire receiving 5V is **Pin 1 of the Plantower PMS5003** connected directly to the USB VBUS pin.





3. Bench Setup & Flashing Procedures

Stage 1: Verify Hardware Detection on Windows

Connect the ShrikeFi board to your laptop using a USB-C data cable. In PowerShell:

```
# Check which COM port is assigned
Get-WmiObject Win32_SerialPort | Select-Object DeviceID, Description
# Expected: "COM4" or "COM5" (Silicon Labs CP210x or USB JTAG/serial debug unit)
```

Stage 2: Compile & Flash Firmware (ESP-IDF v5.5.5)

```
# 1. Export ESP-IDF build environment
. C:\Espressif\frameworks\esp-idf-v5.5.5\export.ps1

# 2. Change to ShrikeFi firmware directory
cd firmware\shrikefi

# 3. Clean, compile, flash, and launch serial monitor
idf.py -p COM4 flash monitor
```

Stage 3: What Legitimate Silicon Boot Output Looks Like

When the board boots successfully, the serial console displays:

```
I (812) WEB_STATUS: Status page up: join WiFi 'VALOR-0081' (open), then open http://192.168.4.1
I (832) LOCATION: Stored location: 'Village Hulimavu, Kolar'
I (832) SHRIKEFI_MAIN: Emergency assist ready (state: IDLE), location 'Village Hulimavu, Kolar'
I (850) SHRIKEFI_LINK: Programming ForgeFPGA bitstream over SPI2 (46408 bytes)...
I (860) SHRIKEFI_LINK: ForgeFPGA configuration COMPLETE (DONE pin asserted HIGH)
I (862) SHRIKEFI_MAIN: MAX30102 initialized OK (100 Hz sampling, FIFO rollover enabled)
I (872) SHRIKEFI_MAIN: BME280 initialized OK at 0x76
I (882) SHRIKEFI_MAIN: PMS5003 initialized OK on UART1 (RX=GPIO14)
I (892) SHRIKEFI_MAIN: MPU6050 6-Axis IMU initialized OK (fall detection active)
I (902) SHRIKEFI_MAIN: L89HS GNSS initialized OK on UART2 (RX=GPIO16)
I (912) SHRIKEFI_MAIN: Core 0: PPG Accelerator Task Started (FPGA hardware link active).
I (922) SHRIKEFI_MAIN: Core 1: Disaster Risk Engine & TinyML Task Started.
[TELEMETRY] NO_FINGER,TEMP=25.4,HUM=52.0,PM25=14.0,SKINTEMP=0.0
```

(Press `Ctrl + ]` to exit the serial monitor before launching PC dashboards).

---

4. Step-by-Step Live Demonstration Script

Act 1: The Offline Architecture & System Boot (0:00 - 1:00)

1. **Physical Action:** Show the unpowered ShrikeFi board to the judges. Point to the Renesas ForgeFPGA chip and the ESP32-S3 SoC.
2. **Spoken Dialogue:** > *"Judges, in an active flood or cyclone disaster, the cloud does not exist. We have zero cellular connectivity and zero server backhaul. This is Valor. It is a dual-brain architecture: a Renesas ForgeFPGA executing optical signal conditioning with 20-nanosecond hardware timing, and a dual-core ESP32-S3 executing clinical triage and a zero-cloud integer neural network."*
3. **Physical Action:** Plug in the USB cable. Point out the OLED display lighting up with live sensor telemetry and the blue LED (D12) on the board.

Act 2: Hardware Jitter Elimination & Live Optical PPG (1:00 - 2:30)

1. **Physical Action:** Place a finger firmly on the MAX30102 pulse oximeter sensor.
2. **Observe Hardware:**
   - The blue LED (D12) flashes in synchrony with every systolic arterial pulse.
   - The OLED display begins showing live Heart Rate (e.g., `HR: 74 BPM`), Blood Oxygenation (`SpO2: 98%`), and Respiratory Rate (`RR: 15`).
3. **Launch Desktop Oscilloscope:** Double-click `launch_dashboard.bat` (or open `firmware/shrikefi/dashboard/valor_dashboard.html` in Chrome and click **Connect Serial** → select **COM4**).
4. **Observe GUI:**
   - The green 60 FPS oscilloscope displays arterial blood volume pulses.
   - Show the judges the Signal Quality Index gauge reading **SQI: 92%** (Hospital Grade).
5. **Spoken Dialogue:** > *"Notice the blue LED flashing on the board. That LED is not driven by software interrupts or CPU code—it is driven directly by the output pin of our FPGA peak detector. In software, RTOS scheduling jitter introduces up to 10 milliseconds of error, which corrupts Heart Rate Variability. Our FPGA detects peaks with 20-nanosecond cycle precision, giving us the mathematical fidelity needed for medical-grade RMSSD calculation."*

Act 3: Multi-Hazard Fusion & Atmospheric Storm Tracking (2:30 - 4:00)

1. **Physical Action:** Point to the BME280 sensor and the barometric pressure trend gauge on the web dashboard.
2. **Spoken Dialogue:** > *"A fitness watch only tells you your heart rate. But what happens if a tropical cyclone is deepening overhead? Valor tracks atmospheric pressure changes across a 180-minute rolling regression window. If the barometer drops faster than 1 hPa per hour, the system automatically elevates the cyclone risk level and warns the user to move inland, independent of any weather broadcast."*
3. **Physical Action:** Lightly blow across the MAX30102 sensor to demonstrate skin temperature tracking. The display transitions from ambient room proxy to human finger contact temperature (`SKINTEMP: 35.8°C`).

Act 4: Fall Detection & Emergency Assistance Latching (4:00 - 5:30)

1. **Physical Action:** Pick up the breadboard containing the MPU6050 IMU. Simulate a realistic fall: drop the board from a height of ~15 cm onto a soft surface (or execute a rapid downward drop followed by a sharp stop and 1.5 seconds of immobility).
2. **Observe System Response:**
   - The OLED immediately transitions to: `*** FALL DETECTED ***`.
   - The system enters `ARMED` state with a 10-second countdown timer.
3. **Spoken Dialogue:** > *"Our 3-stage biomechanical fall detection state machine just registered free-fall weightlessness, impact shock, and postural immobility. It is now in a 10-second cancellation window to prevent false alarms if the user simply dropped their bag."*
4. **Physical Action:** Allow the 10 seconds to expire without pressing cancel.
5. **Observe Emergency Escalation:**
   - The OLED switches to the full-screen **Emergency Assistance Card**:

     ```
     * MEDICAL EMERGENCY *
     SOS ACTIVE  T+00:15
     HR: 76  SpO2: 98%
     RR: 15  SQI:  92%
     COND: CRITICAL FALL
     LOC: Village Hulimavu
     CALL 108 DO NOT MOVE
     ```

   - The active sounder triggers an audible siren.

Act 5: Zero-Install Local Rescuer Web Dashboard (5:30 - 7:00)

1. **Physical Action:** Take out your smartphone. Open Wi-Fi settings.

2. **Observe SSID:** Show the judges the open Wi-Fi network: **`VALOR-0081`**.
3. **Connect & Browse:** Connect the phone to `VALOR-0081` (no password required). Open the phone browser and navigate to:

   `http://192.168.4.1`

4. **Show Phone Screen to Judges:**
   - The phone renders the responsive, dark-mode VALOR dashboard served directly from the ESP32-S3 flash memory.
   - The top banner flashes crimson: **`MEDICAL EMERGENCY: SOS ACTIVE (fall)`**.
   - Show the live vitals, environmental metrics, and the stored geolocation (`Village Hulimavu, Kolar`).
5. **Remote Cancellation:** Tap the **CANCEL SOS** button on the phone screen. The device immediately clears the emergency state, and the OLED returns to normal vital monitoring.
6. **Spoken Dialogue:** > *"Judges, notice that my phone did not download an app from the App Store, and there is no router in this room. The device itself is an autonomous web server. Any first responder who arrives on the scene can connect their phone in seconds and access full patient vitals, 24-hour trends, and location data."*

---

5. Real Hardware Verification vs. Verified Fallback Demos

In competitive hackathons, physical hardware can suffer broken jumper wires, dead USB hubs, or hostile RF interference. Valor includes **verified fallback modes** that run on standard laptops without physical silicon.

Comparison Matrix: What Proves What

Feature	Primary Proof (Hardware Silicon)	Verified Fallback Proof (Host PC Simulation)	What the Fallback Truly Proves
FPGA Peak Detector	Physical Blue LED (D12) flashes to real pulse	`tb_forgefpga_system.v` simulation in Icarus Verilog	Proves 100% cycle-accurate Verilog logic and 20 ns timing closure
Zero-Cloud INT8 AI	ESP32-S3 runs inference in 18 µs on live vitals	`shrikefi_host.exe` host test suite under GCC 15.2	Proves mathematical quantization fidelity (max error 0.091)

Feature	Primary Proof (Hardware Silicon)	Verified Fallback Proof (Host PC Simulation)	What the Fallback Truly Proves
mNEWS2 Triage Engine	OLED displays live clinical levels and alerts	`test_disaster_risk_engine.exe` (21 unit tests)	Proves all clinical boundary sweeps and crisis overrides
Fall Detection FSM	MPU6050 physical drop triggers countdown	`fall_test.exe` (16 FSM unit tests)	Proves 3-stage FSM state transitions and angle math
GUI & Triage Dashboard	Live Web Serial stream on COM4 port	`valor_dashboard.html` with built-in Simulation Profiles	Proves UI rendering, gauge math, and multi-hazard response

Executing the Verified Fallback Demonstration (If Hardware Fails)

If a sensor wire breaks or your board loses power, do **not** panic. Immediately transition to the verified software and simulation fallback:

1. **Launch the Web Dashboard in Simulation Mode:** Double-click `launch_web_dashboard.bat`. Chrome opens `valor_dashboard.html`.
2. **Select a Disaster Profile:** Under **Simulated Profiles**, select:
   - **`Heat Wave & Dehydration`**: Watch HR climb to 138 BPM, Ambient Temp to 43.5°C, Moran's PSI climb to 9.5, and the TinyML Heat Stroke gauge hit 94% (CRITICAL).
   - **`Severe Smog / PM2.5 Crisis`**: Watch PM2.5 rise to 340 µg/m³, SpO2 drop to 92.5%, and the Smog Inhalation gauge spike to 97%.
   - **`Flash Flood / Hypothermia`**: Watch ambient temperature plunge to 11.2°C, barometric slope drop to -3.8 hPa/hr, triggering the **CRITICAL CYCLONE** warning card.
3. **Execute Verilog Hardware Simulation in Front of Judges:** Open PowerShell and run:

   ```
   cd hardware\shrikefi
   iverilog -o sim.vvp -I ../common tb_forgefpga_system.v forgefpga_ppg_top.v spi_target.v ../commo
   vvp sim.vvp
   ```

   Show the judges the terminal output: `RESULTS: 10 passed, 0 failed (out of 10 checks) - ALL TESTS PASSED.`

---

6. Troubleshooting Guide for Live Stage Failures

Symptom / Error	Immediate Root Cause	30-Second Fix
`Permission denied: build_host_tests.exe`	A previously compiled test executable is still running in the background	Run `Stop-Process -Name *host_test* -Force` in PowerShell, then recompile.
OLED display is completely black	I2C pull-up collision or loose SDA/SCL wire	Jiggle breadboard rows 4 and 5 (GPIO 1 & 2). Power-cycle USB cable.
`[TELEMETRY] NO_FINGER` permanently displayed	MAX30102 photodiode saturated by overhead lighting	Cup your hand over the sensor to shield it from harsh fluorescent lights. Warm your finger.
Phone cannot connect to `VALOR-0081`	Wi-Fi channel congestion or phone auto-disconnecting (no internet)	Disable mobile data on the phone; hold phone within 1 meter of the ESP32-S3 board.
Web Serial reports `Failed to open COM port`	Another terminal (e.g., `idf.py monitor`) has locked the serial port	Close all active serial consoles or PowerShell windows running `idf.py monitor`.

---

7. Pre-Judging 15-Minute Readiness Checklist

Complete this checklist 15 minutes before your scheduled judging slot:

- ☐ **USB Cables Tested:** Both USB-C cables confirmed working for data communication.
- ☐ **Firmware Flashed:** Board has the latest firmware build with boot banner verified.
- ☐ **COM Port Verified:** Note whether your device is on COM4, COM5, or COM3.
- ☐ **Phone Wi-Fi Tested:** Phone successfully connects to `VALOR-0081` and loads `http://192.168.4.1`.
- ☐ **Browser Tabs Prepared:**
  - Tab 1: `firmware/shrikefi/dashboard/valor_dashboard.html` (Web Serial Dashboard).
  - Tab 2: `hardware/shrikefi/synthesis_evidence/resource_utilization_spi_link.log` (FPGA Evidence).
  - Tab 3: `docs/PROJECT_GUIDE.md` (Architecture Reference).
- ☐ **Terminal Prepared:** PowerShell open in repo root with GCC compiler and Icarus Verilog verified on PATH.

☐ **Sensors Secured:** Sensor breakout boards firmly seated in breadboard rows with no loose floating wires.

☐ **Spoken Roles Assigned:**

- Teammate 1: Live Hardware Demonstration & Sensor Manipulation.
- Teammate 2: RTL & Hardware Architecture Explanation.
- Teammate 3: Clinical Triage & Zero-Cloud TinyML Model Explanation.